

Projeto de Telecomuni- cações e Informática II

SERVIÇOS CORPORATIVOS DE MENSAGENS E ALERTAS
PARA REDES VEICULARES

FASE 5

Mestrado Integrado em Engenharia de Telecomunicações e Informática

Grupo 1:

- Leandro Henrique Dantas Alves A82157
- André Martins Almeida A82211
- José Eduardo da Silva Santos..... A82350

UNIVERSIDADE DO MINHO | ANO LETIVO 2019/2020



Índice

Índice	2
Índice de figuras	4
Índice de tabelas.....	6
1 Introdução	7
2 Rede	7
2.1 Topologia.....	7
2.2 Endereçamento IPv6	8
2.3 Encaminhamento	10
3 IMTT	14
3.1 Base de Dados.....	14
3.1.1 Veículos.....	15
3.1.2 Alertas.....	16
3.2 Serviço de Resolução de Nomes	17
4 Funcionalidades gerais	20
4.1 Bases de Dados	20
4.1.1 Veículos.....	20
4.1.2 Alertas.....	22
4.1.3 Eventos	24
4.2 Certificados e encriptação	25
4.2.1 Registo de veículos	26
4.3 Protocolo de comunicação	28
4.3.1 Registo de veículos	28

4.3.2	Alertas.....	28
4.3.3	Mensagens de serviços.....	29
4.4	RSU's	31
4.5	OBU's – Veículos	33
4.5.1	Serviço de localização geográfica e de estado <i>online</i>	34
4.5.2	Alertas.....	35
5	Marca Buffalo #1.....	37
5.1	Monitorização de veículos	37
5.2	Troca de certificados.....	38
5.3	Serviços	40
5.3.1	Alertas de manutenção	40
5.3.2	Partilha de localização	43
5.3.3	Chamada de voz.....	47
6	Marca Cheetah #2.....	51
6.1	Monitorização de veículos	51
6.2	Serviços	52
6.2.1	<i>Geocoding</i> – informação da localização	52
6.2.2	Alertas de baixo combustível e posto mais próximo	54
6.2.3	Pedidos de SOS	57
7	Funcionalidades planeadas não implementadas	62
7.1	<i>Mobility script</i>	63
8	Software utilizado.....	63
9	Conclusão	64
10	Referências	65

Índice de figuras

Figura 2.1 – Topologia da rede.....	7
Figura 2.2 – Configuração da WLAN.....	8
Figura 2.3 – Endereçamento IPv6 de um OBU baseado no MAC.....	10
Figura 2.4 – Configuração do serviço <i>zebra</i> do <i>router</i> central da rede IMTT.	11
Figura 2.5 – Configuração do serviço <i>zebra</i> de um dos <i>routers</i> BGP da rede IMTT.	12
Figura 2.6 – Configuração do serviço <i>zebra</i> de um dos RSU's da rede Buffalo.....	13
Figura 3.1 – Aplicação do <i>host</i> do IMTT.	14
Figura 3.2 – Ficheiro de armazenamento dos endereços de cada marca.....	15
Figura 3.3 – Lista de veículos registados pelo IMTT.	15
Figura 3.4 – Alerta lançado e listado pelo IMTT.....	17
Figura 3.5 – Ficheiro de atribuição de DNS.	18
Figura 3.6 – <i>Ping</i> a um endereço atribuído.	18
Figura 3.7 – Configuração do arranque automático dos serviços de BIND e MySQL no <i>host</i> da rede IMTT.....	19
Figura 4.1 – Lista de veículos registados pela marca.	21
Figura 4.2 – Registo de veículo feito pela marca com atribuição de DNS pelo IMTT.	22
Figura 4.3 – Diagrama de sequência do lançamento de um alerta pelo IMTT.	23
Figura 4.4 – Alerta do IMTT recebido e listado pela marca.	23
Figura 4.5 – Lista de eventos armazenados.	25
Figura 4.6 – Certificados armazenados no <i>host</i> da marca.	26
Figura 4.7 – <i>Keystore</i> armazenada por um OBU.	27
Figura 4.8 – Diagrama de sequência do registo de um veículo.....	27
Figura 4.9 – Fluxograma da receção de mensagens pelos OBU's.....	30
Figura 4.10 – Diagrama de sequência do envio de mensagens do <i>host</i> da marca para a rede veicular via RSU's.....	31
Figura 4.11 – Diagrama de sequência da receção de mensagens do <i>host</i> da marca enviadas pela rede veicular via RSU's.....	32
Figura 4.12 – Diagrama de sequência da devolução de mensagens destinadas a veículos para a rede veicular.	32
Figura 4.13 – Diagrama de sequência da devolução de mensagens de outras marcas para a rede veicular.	32
Figura 4.14 – Sinalização das mensagens passadas por um RSU.	33
Figura 4.15 – Diagrama de sequência da obtenção de coordenadas GPS e envio de <i>keep alive</i>	34
Figura 4.16 – Coordenadas GPS e alerta ativo mais recente visíveis na aplicação veicular.....	35
Figura 4.17 – Esquema das zonas de alerta e de notificação.....	36
Figura 4.18 – Lista de alertas ativos no OBU.....	36
Figura 5.1 – Monitorização de veículos da marca Buffalo.	37

Figura 5.2 – Diagrama de sequência do pedido, troca e armazenamento de certificados.....	39
Figura 5.3 – Lista de certificados armazenados por três OBU's.....	40
Figura 5.4 – Agendamento e registo de manutenção de um veículo.	41
Figura 5.5 – Alerta no OBU de data de manutenção a aproximar-se.	42
Figura 5.6 – Alerta no OBU de data de manutenção ultrapassada.....	42
Figura 5.7 – Diagrama de sequência do envio de alertas de manutenção.	43
Figura 5.8 – OBU à espera da aceitação do pedido de partilha de localização.	44
Figura 5.9 – Pedido de partilha de localização recebido por OBU.....	45
Figura 5.10 – Partilha de localização feita entre dois OBU's.....	45
Figura 5.11 – Diagrama de sequência do pedido, aceitação e partilha de localização.	46
Figura 5.12 - OBU à espera da aceitação do pedido de chamada.....	48
Figura 5.13 – Pedido de chamada recebido por OBU.	48
Figura 5.14 – Diagrama de sequência do pedido, aceitação e funcionamento da chamada.....	49
Figura 5.15 – Intensidade de um pacote de áudio enviado por um OBU.....	50
Figura 5.16 – Intensidade de um pacote de áudio recebido por um OBU.....	50
Figura 6.1 – Monitorização de veículos da marca Cheetah.	51
Figura 6.2 – Diagrama de sequência da determinação e envio do <i>geocode</i>	53
Figura 6.3 – <i>Geocode</i> visualizado na aplicação veicular.....	54
Figura 6.4 – Diagrama de sequência da obtenção do posto de combustível mais próximo para concatenar ao <i>geocode</i>	55
Figura 6.5 – Posto de combustível mais próximo apresentado na aplicação veicular.	56
Figura 6.6 – Combustível vazio indicado na aplicação veicular.	57
Figura 6.7 – Diagrama de sequência do alerta de acidente lançado pelo IMTT após pedido da marca.	58
Figura 6.8 – Pedido de SOS feito pelo OBU com alerta recebido pelo IMTT.	59
Figura 6.9 – Notificação de alerta IMTT num veículo nas proximidades do acidente.	60
Figura 6.10 – Pedido de SOS (e cancelamento) e alerta IMTT recebidos pela marca.	61
Figura 8.1 – Mapa da topologia secundária.	64

Índice de tabelas

Tabela 2.1 – Endereçamento das sub-redes.	8
Tabela 2.2 – Endereçamento externo BGP.	9
Tabela 2.3 – Endereçamento interno da rede IMTT.	9
Tabela 2.4 – Endereçamento interno da rede Buffalo.	9
Tabela 2.5 – Endereçamento interno da rede Cheetah.	9
Tabela 2.6 – Endereçamento WLAN dos RSU's.	9
Tabela 4.1 – Estrutura dos eventos gerais armazenados na tabela de eventos....	24
Tabela 5.1 – Estrutura do registo e agendamento de manutenção armazenados na tabela de eventos.....	41
Tabela 6.1 – Estrutura do alerta de combustível vazio armazenado na tabela de eventos.....	56
Tabela 6.2 – Estrutura do pedido de SOS armazenado na tabela de eventos.	58

1 Introdução

Neste relatório estão demonstradas e descritas todas as funcionalidades deste projeto, incluindo a topologia, configuração dos seus nós, das suas funções e endereços, de todos os serviços implementados com as razões das suas estruturas. Também estão descritas as limitações que houve, assim como alguns serviços que apesar de planeados, não foram implementados.

2 Rede

Através do *software* CORE [1], foi emulada uma rede IPv6 (*Internet Protocol version 6*) a partir da gama 2001:0690:2280:820::0/61, sendo constituída por sub-redes de IMTT, marcas (Buffalo [2] e Cheetah [3]) e veicular.

2.1 Topologia

Como está demonstrada na Figura 2.1, a topologia da rede inclui uma rede IMTT constituída por um *host* (n23) ligado por um *switch* (n24) a um *router* central (n1) ligado a *routers* BGP (*Border Gate Protocol*) (n2 e n3) para comunicarem com as redes das marcas.

As redes de cada marca (Buffalo^B e Cheetah^C) também contém *hosts* (n12^B e n13^C) ligados via *switch* (n14^B e n15^C) ao respetivo *router* central (n6^B e n7^C), que por sua vez está ligado a *routers* BGP (n4^B e n5^C) que ligam à rede IMTT.

Cada marca tem também ligado ao seu *router* central um *switch* (n30^B e n11^C) ligado a um número variável de MDR's (*Modular Data Routers*) a servir como RSU's (*Roadside Units*).

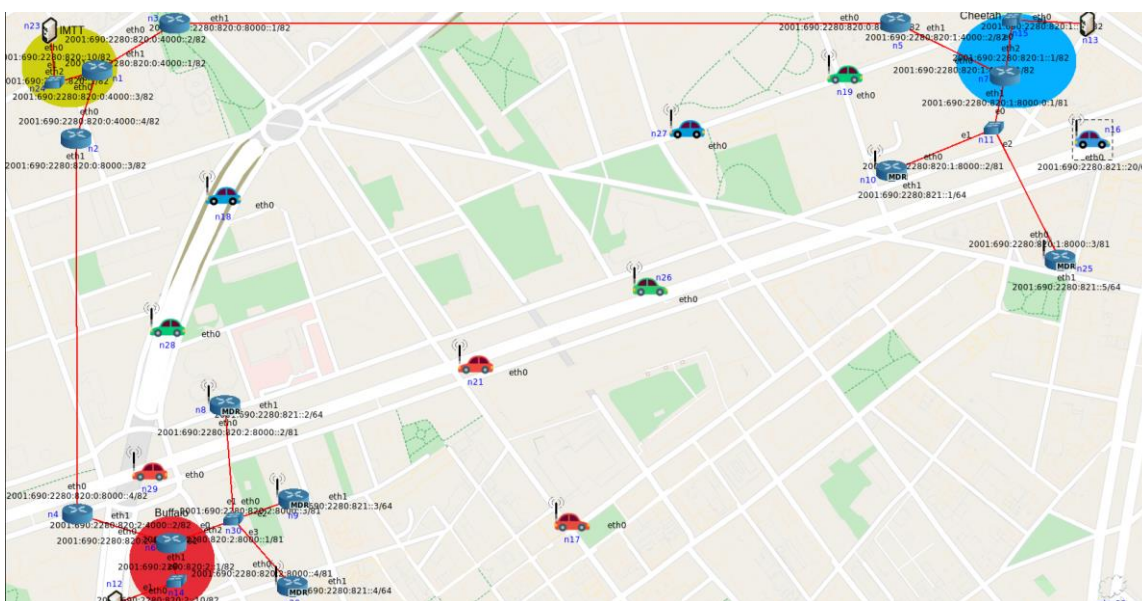


Figura 2.1 – Topologia da rede.

Foi também aplicada uma rede veicular WLAN (*Wireless Local Area Network*) (wlan22), configurada de acordo com a Figura 2.2, como meio de comunicação entre os veículos compostos por OBU's (*Onboard Units*), sendo os RSU's parte da mesma a servir como meio de comunicação entre as OBU's e as redes de marca.

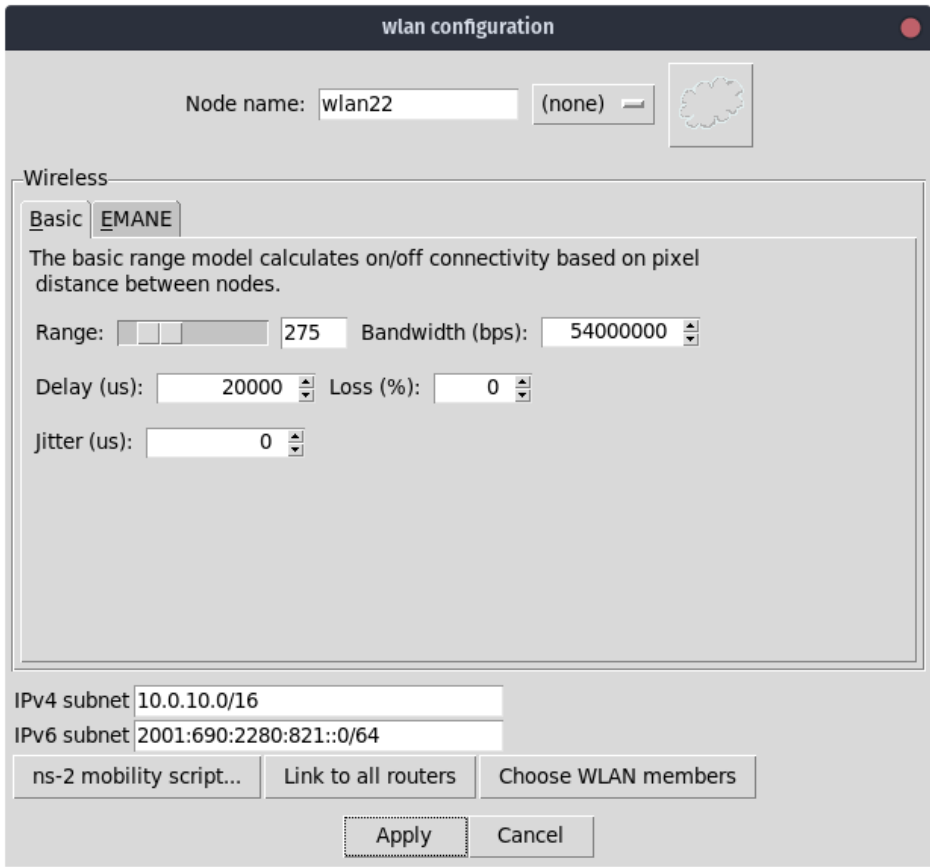


Figura 2.2 – Configuração da WLAN.

2.2 Endereçamento IPv6

Sendo * = 2001:690:2280, na Tabela 2.1 encontra-se o endereçamento IPv6 feito para a gama e os *hosts* de cada rede.

Tabela 2.1 – Endereçamento das sub-redes.

Rede	Endereço de rede	Host	DNS
IMTT	*:820::0/80	*:820::10/82	imtt.auto.pt
Buffalo	*:820:2::0/80	*:820:2::10/82	buffalo.auto.pt
Cheetah	*:820:1::0/80	*:820:1::10/82	cheetah.auto.pt
Veicular (WLAN)	*:821::0/64		

O endereçamento das interligações externas BGP dos *routers* está indicado na Tabela 2.2.

Tabela 2.2 – Endereçamento externo BGP.

<i>Router 1</i>	<i>Interface 1/máscara</i>	<i>Interface 2</i>	<i>Router 2</i>
BGP IMTT (n2)	*:820:0:8000::3/82	*:820:0:8000::4	BGP Buffalo (n4)
BGP IMTT (n3)	*:820:0:8000::1/82	*:820:0:8000::2	BGP Cheetah (n5)

Para o endereçamento interno da rede IMTT, foram usados os endereços da Tabela 2.3.

Tabela 2.3 – Endereçamento interno da rede IMTT.

<i>Router 1</i>	<i>Interface 1/máscara</i>	<i>Interface 2</i>	<i>Router 2</i>
Central (n1)	*:820::1/82	*:820::10/82	Host (n23)
	*:820:4000::3/82	*:820:4000::4	BGP (n2)
	*:820:4000::1/82	*:820:4000::2	BGP (n3)

No caso dos endereçamentos internos das redes Buffalo e Cheetah, o endereçamento foi feito de acordo com a Tabela 2.4 e a Tabela 2.5, respetivamente.

Tabela 2.4 – Endereçamento interno da rede Buffalo.

<i>Router 1</i>	<i>Interface 1/máscara</i>	<i>Interface 2</i>	<i>Router 2</i>
Central (n6)	*:820:2::1/82	*:820:2::10	Host (n12)
	*:820:2:4000::1/82	*:820:2:4000::2	BGP (n4)
	*:820:2:8000::1/81	*:820:2:8000::2	RSU (n8)
		*:820:2:8000::3	RSU (n9)
		*:820:2:8000::4	RSU (n20)

Tabela 2.5 – Endereçamento interno da rede Cheetah.

<i>Router 1</i>	<i>Interface 1/máscara</i>	<i>Interface 2</i>	<i>Router 2</i>
Central (n7)	*:820:1::1/82	*:820:1::10	Host (n13)
	*:820:1:4000::1/82	*:820:1:4000::2	BGP (n5)
	*:820:1:8000::1/81	*:820:1:8000::2	RSU (n10)
		*:820:1:8000::3	RSU (n25)

Na rede veicular, os endereços atribuídos aos RSU's foram os da Tabela 2.6.

Tabela 2.6 – Endereçamento WLAN dos RSU's.

<i>Router</i>	<i>Endereço</i>
RSU Buffalo (n8)	*:821::2/64
RSU Buffalo (n9)	*:821::3/64
RSU Buffalo (n20)	*:821::4/64
RSU Cheetah (n10)	*:821::1/64
RSU Cheetah (n25)	*:821::5/64

Os endereços atribuídos automaticamente aos veículos são baseados nos seus endereços MAC (*Media Access Control*), de acordo com o Processo EUI-64 (*Extended Unique Identifier 64*) [4], como está exemplificado na Figura 2.3 (Endereço MAC: 00:00:00:aa:00:0c, IPv6: *:824:200:ff:feaa:c).

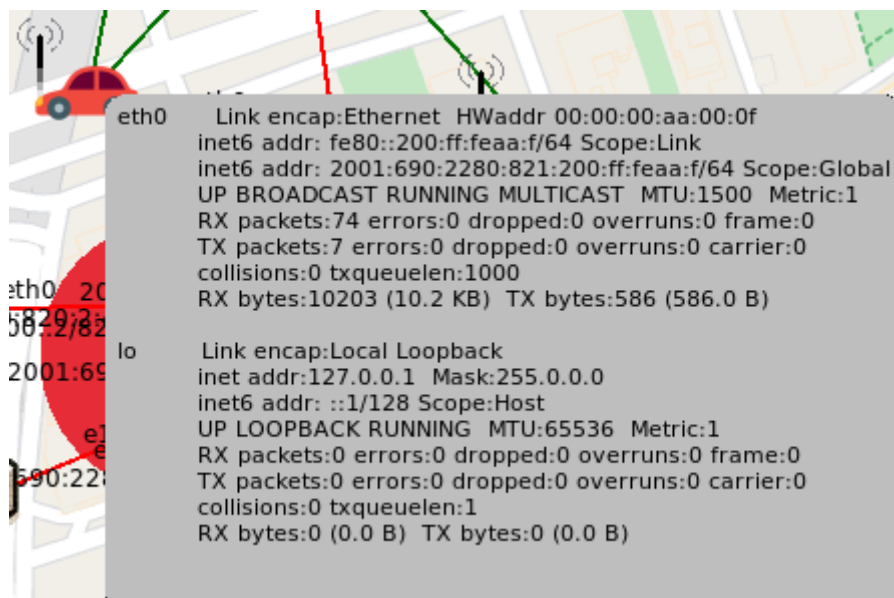


Figura 2.3 – Endereçamento IPv6 de um OBU baseado no MAC.

2.3 Encaminhamento

Com apoio da documentação do Quagga [5], foi configurado o encaminhamento em todos os *routers*.

Para que os pacotes enviados pelos *hosts* ou pelos RSU's fossem encaminhados entre os *routers* no caminho, foi configurado o encaminhamento OSPF (*Open Shortest Path First*) nos *routers* centrais.

Na Figura 2.4 encontra-se a implementação do OSPFv6 no serviço *zebra* do *router* central do IMTT (n1) para todas as suas interfaces.

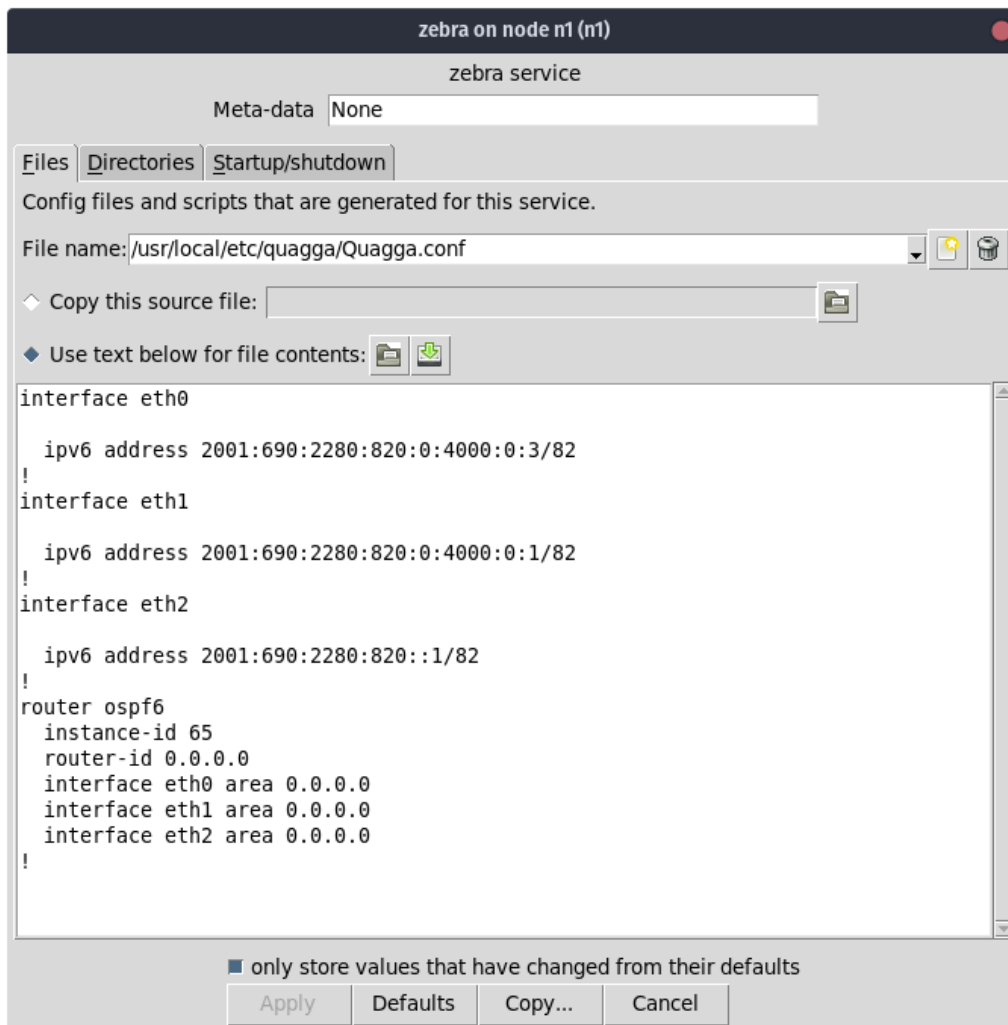


Figura 2.4 – Configuração do serviço *zebra* do *router* central da rede IMTT.

Esta configuração é semelhante à dos *routers* centrais das marcas (n6 e n7), sendo a única diferença das marcas a adição da linha “redistribute ospf” para que as mensagens de RSU’s possam ser reencaminhadas para outros RSU’s.

Para os *routers* BGP do IMTT, foi ativado o serviço BGP e foram feitas as configurações da Figura 2.5 no serviço *zebra* (n3).

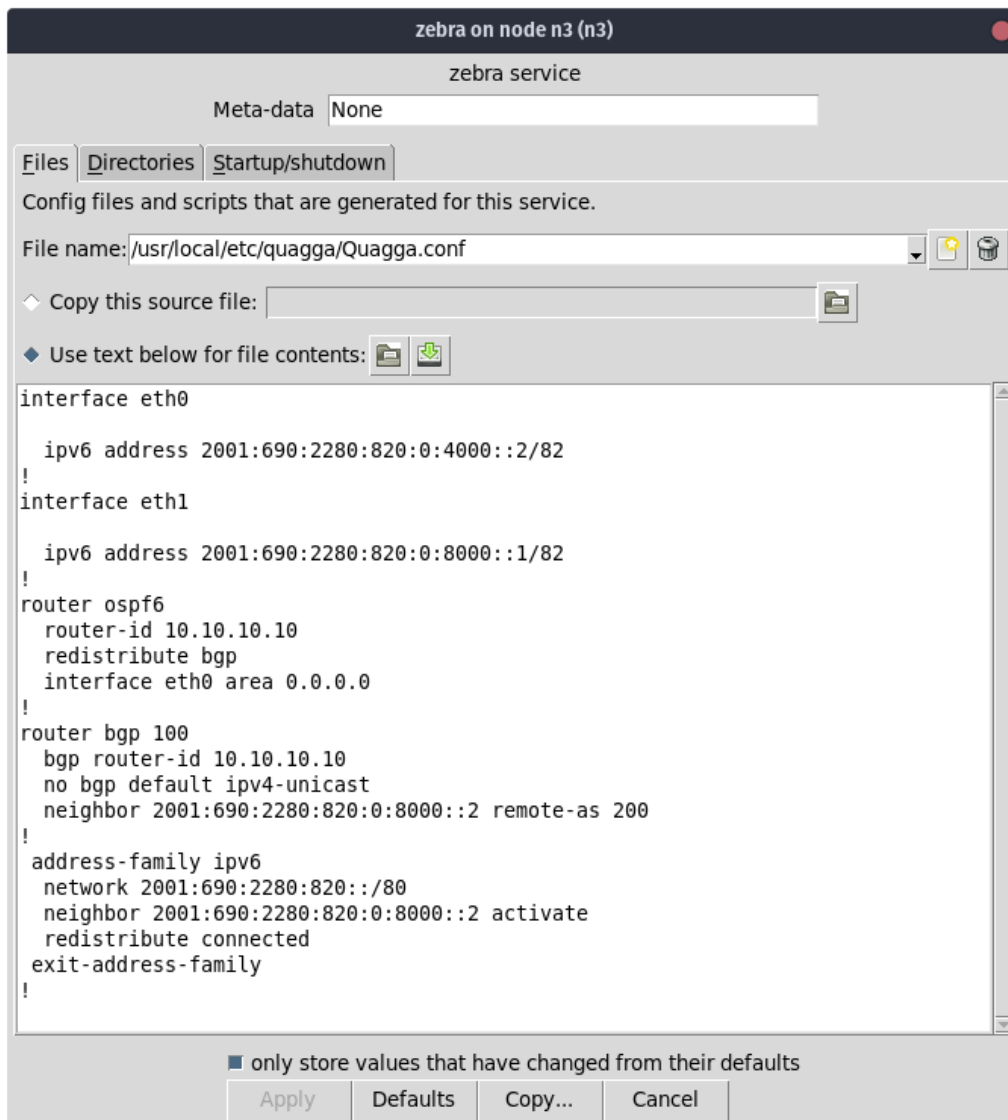


Figura 2.5 – Configuração do serviço *zebra* de um dos *routers* BGP da rede IMTT.

Nesta configuração, foram definidos o identificador do *router* (100) e o endereço e identificador (200) da interface do *router* vizinho (n5, BGP Cheetah). As configurações dos restantes *routers* BGP (n2, n4 e n5) são idênticas, mas com os respectivos identificadores, endereços e vizinhos.

Para os RSU's das marcas, foi feita a configuração da Figura 2.6 (n8). O OSPFv6 é também aplicado, assim como o meio de atribuição de endereço IPv6 aos veículos que estiverem no seu alcance.

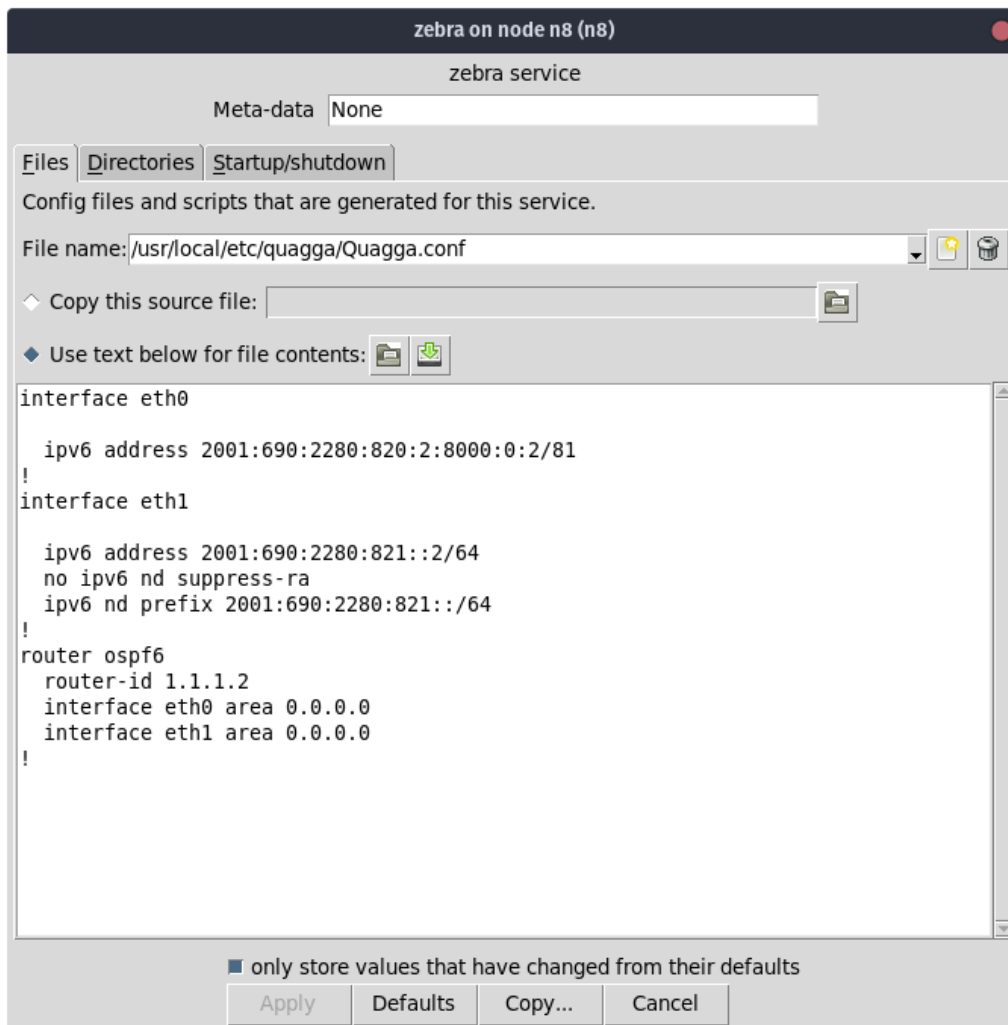


Figura 2.6 – Configuração do serviço *zebra* de um dos RSU's da rede Buffalo.

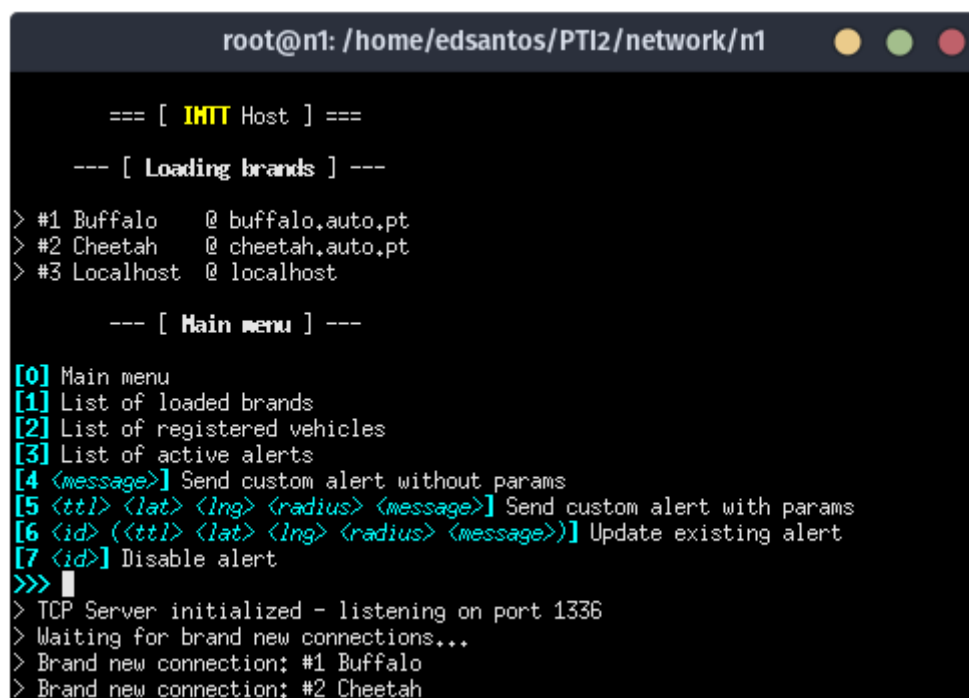
No caso dos OBU's dos veículos, o encaminhamento de pacotes é feito em *multicast* para o endereço FF02::1 (assim como os RSU's, se receberem uma mensagem que deve ser reencaminhada para um veículo, independentemente de vir do *host* da marca ou mesmo de um veículo). Este encaminhamento é feito por UDP (*User Datagram Protocol*); contudo, entre RSU's e o *host* da marca é usado TCP (*Transmission Control Protocol*) numa arquitetura cliente/servidor.

3 IMTT

Foi desenvolvido o Serviço IMTT no *host* da Rede IMTT, hospedando uma base de dados, um servidor para o serviço DNS (*Domain Name System*) e um sistema de envio automático e manual de alertas gerais.

O papel do *host* IMTT na arquitetura da rede é de servir de servidor TCP *multi-thread* para os *hosts* das marcas, havendo uma *thread* criada para cada conexão de uma marca, de modo a ter uma comunicação assíncrona com todas as marcas.

O *software* aplicacional foi desenvolvido em Java, estando o seu *snapshot* na Figura 3.1.

A terminal window titled 'root@n1: /home/edsantos/PTI2/network/n1' displays the IMTT Host application. The interface shows a loading phase for brands, listing Buffalo, Cheetah, and Localhost. It then presents a main menu with options like listing brands, vehicles, alerts, and sending alerts. The terminal shows the application is listening on port 1336 and has received connections from Buffalo and Cheetah.

```
root@n1: /home/edsantos/PTI2/network/n1

=== [ IMTT Host ] ===

--- [ Loading brands ] ---
> #1 Buffalo    @ buffalo.auto.pt
> #2 Cheetah    @ cheetah.auto.pt
> #3 Localhost  @ localhost

--- [ Main menu ] ---

[0] Main menu
[1] List of loaded brands
[2] List of registered vehicles
[3] List of active alerts
[4] <message>] Send custom alert without params
[5] <ttl> <lat> <lng> <radius> <message>] Send custom alert with params
[6] <id> (<ttl> <lat> <lng> <radius> <message>)] Update existing alert
[7] <id>] Disable alert
>>>
> TCP Server initialized - listening on port 1336
> Waiting for brand new connections...
> Brand new connection: #1 Buffalo
> Brand new connection: #2 Cheetah
```

Figura 3.1 – Aplicação do *host* do IMTT.

3.1 Base de Dados

No *host* da rede IMTT há uma instância MySQL a correr, alojando uma base de dados IMTT para registo de veículos e alertas.

3.1.2 Alertas

Para armazenamento de alertas e monitorização dos ativos, existe uma tabela **Alert** preenchida ao ser lançado um novo alerta pelo IMTT, tendo as seguintes colunas:

- **id** (chave primária, auto-incrementável) – identificador numérico do alerta;
- **tstamp** – *timestamp* da inserção do alerta;
- **tvl** – *timestamp* da validade do alerta;
- **latitude** – latitude (em graus) do epicentro do alerta;
- **longitude** – longitude (em graus) do epicentro do alerta;
- **radius** – raio de alcance (em metros) do alerta a partir do seu epicentro;
- **message** – descrição do alerta.

Estes alertas, ao serem lançados, são enviados para as marcas conectadas, devendo estas encaminhá-los para a rede veicular.

Em alertas que têm os parâmetros de latitude, longitude e raio com valores = 0 (valor padrão, caso não sejam preenchidos estes parâmetros em alertas manuais), em vez de se pretender alertar apenas os veículos cujas coordenadas encontram-se a uma distância inferior ao raio de alcance definido do epicentro, estes alertas são destinados a todos os veículos, independentemente da localização; útil, por exemplo, para anúncios.

Alertas existentes também podem ter os seus campos atualizados, selecionando o ID do alerta desejado. Para cancelar, TTL (*Time To Live*) = *timestamp* atual.

Na Figura 3.4 encontra-se um exemplo de lançamento e listagem dum alerta com a mensagem “Traffic” como único parâmetro.

```
root@n1: /home/edsantos/PTI2/network/n1
[5 <ttl> <lat> <lng> <radius> <message>] Send custom alert with params
[6 <id> (<ttl> <lat> <lng> <radius> <message>)] Update existing alert
[7 <id>] Disable alert
>>> 2
> TCP Server initialized - listening on port 1336
> Waiting for brand new connections...
> Brand new connection: #1 Buffalo
> Brand new connection: #2 Cheetah

--- [ List of registered vehicles ] ---
> P 66-66-RR 20/06 | Buffalo

>>> 4 Traffic
> Sending alert: #103 Traffic at (0,0º, 0,0º) ±0 m until 2020-06-22 11:30:01

>>> 3

--- [ List of active alerts ] ---
> #103 [2020-06-22 11:29:01] Traffic at (0,0º, 0,0º) ±0 m until 2020-06-22 11:30:01

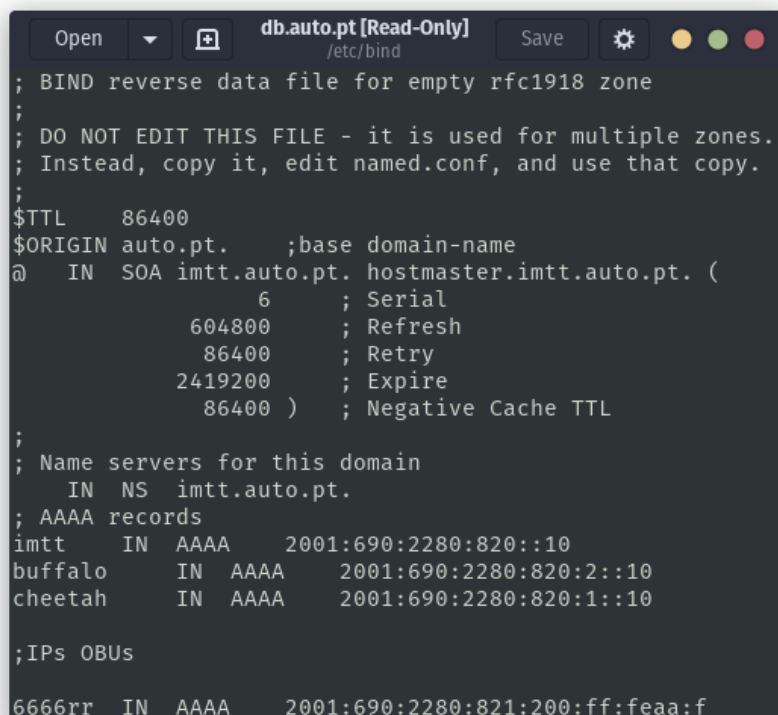
>>> █
```

Figura 3.4 – Alerta lançado e listado pelo IMTT.

3.2 Serviço de Resolução de Nomes

No registo de um veículo, é-lhe atribuído um endereço de DNS com domínio em auto.pt, sendo o prefixo atribuído dependente da matrícula (12-34-SG → 1234sg.auto.pt). Para tal, foi desenvolvido um Serviço de Resolução de Nomes através do *software* BIND 9 [7] e com o apoio de diversos tutoriais [8] [9], para atribuir endereços de DNS aos endereços IPv6 das marcas e do IMTT, e principalmente aos endereços dos veículos.

A configuração deste serviço constituiu na configuração do ficheiro **db.auto.pt**, demonstrada na Figura 3.5.



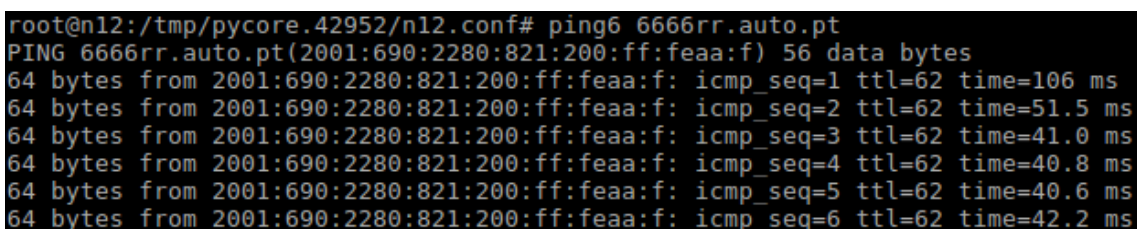
```
; BIND reverse data file for empty rfc1918 zone
;
; DO NOT EDIT THIS FILE - it is used for multiple zones.
; Instead, copy it, edit named.conf, and use that copy.
;
$TTL      86400
$ORIGIN    auto.pt.      ;base domain-name
@   IN     SOA    imtt.auto.pt. hostmaster.imtt.auto.pt. (
                        6      ; Serial
                        604800 ; Refresh
                        86400  ; Retry
                        2419200 ; Expire
                        86400 ) ; Negative Cache TTL
;
; Name servers for this domain
;   IN     NS     imtt.auto.pt.
; AAAA records
imtt   IN     AAAA    2001:690:2280:820::10
buffalo   IN     AAAA    2001:690:2280:820:2::10
cheetah   IN     AAAA    2001:690:2280:820:1::10

;IPs OBU's
6666rr   IN     AAAA    2001:690:2280:821:200:ff:feaa:f
```

Figura 3.5 – Ficheiro de atribuição de DNS.

Na Figura 3.5 encontra-se a lista dos domínios atribuídos a cada endereço IPv6, nomeadamente os endereços dos *hosts* do IMTT (imtt.auto.pt) e das marcas Buffalo (buffalo.auto.pt) e Cheetah (cheetah.auto.pt) e dos veículos registados na Figura 3.3 (formato <matrícula>.auto.pt).

Este funcionamento está comprovado na Figura 3.6 através do comando “ping”.



```
root@n12:/tmp/pycore.42952/n12.conf# ping6 6666rr.auto.pt
PING 6666rr.auto.pt(2001:690:2280:821:200:ff:feaa:f) 56 data bytes
64 bytes from 2001:690:2280:821:200:ff:feaa:f: icmp_seq=1 ttl=62 time=106 ms
64 bytes from 2001:690:2280:821:200:ff:feaa:f: icmp_seq=2 ttl=62 time=51.5 ms
64 bytes from 2001:690:2280:821:200:ff:feaa:f: icmp_seq=3 ttl=62 time=41.0 ms
64 bytes from 2001:690:2280:821:200:ff:feaa:f: icmp_seq=4 ttl=62 time=40.8 ms
64 bytes from 2001:690:2280:821:200:ff:feaa:f: icmp_seq=5 ttl=62 time=40.6 ms
64 bytes from 2001:690:2280:821:200:ff:feaa:f: icmp_seq=6 ttl=62 time=42.2 ms
```

Figura 3.6 – Ping a um endereço atribuído.

Para este serviço ser iniciado automaticamente, foram feitas as seguintes configurações no *host* do IMTT no CORE (comandos “/etc/init.d/bind9 start” e “/etc/init.d/resolvconf start”), demonstradas na Figura 3.7.



Figura 3.7 – Configuração do arranque automático dos serviços de BIND e MySQL no *host* da rede IMTT.

4 Funcionalidades gerais

Há certas funcionalidades e serviços comuns a ambas as marcas.

4.1 Bases de Dados

Ambas as marcas contém uma base de dados para cada uma, contendo três tabelas para armazenamento e monitorização de veículos, alertas e eventos.

Devido às limitações do CORE, a instância MySQL destas bases de dados teve de ser a mesma instância usada pelo IMTT pois só pode haver um serviço de MySQL a correr no sistema operativo, que é usado por todos os nós do CORE.

4.1.1 Veículos

A tabela **Vehicle** de ambas as marcas contém as seguintes colunas:

- **licencePlate** (chave primária) – matrícula do veículo;
- **lastOnline** – *timestamp* da última mensagem do veículo recebida, indicando quando foi a última vez que o veículo esteve ativo/*online*.

As restantes colunas são dependentes dos serviços de cada marca. Para a marca Buffalo, estão contidas as seguintes colunas:

- **lastMaintenance** – *timestamp* da última manutenção feita;
- **nextMaintenance** – *timestamp* da próxima manutenção agendada.

No caso da marca Cheetah, existem as seguintes colunas:

- **latitude** – latitude da localização geográfica do veículo;
- **longitude** – longitude da localização geográfica do veículo;
- **fuel** – percentagem de combustível;
- **sos** – mensagem do pedido de SOS.

Um exemplo de listagem de veículos registados pela marca Buffalo encontra-se na Figura 4.1.

```
root@n2: /home/edsantos/PTI2/network/n2

=== [ Buffalo Host ] ===

> Connecting to IMTT's TCP Server @imtt.auto.pt:1336...
> Successfully connected to IMTT's Server @imtt.auto.pt:1336

--- [ Main menu ] ---

[0] Main menu
[1] List of registered vehicles
[2] List of active IMTT alerts
[3] List of all events
[4] <address> <licence plate> (<country> <year> <month>)] Register vehicle
[5] <licence plate> <yyyy-MM-dd>] Schedule vehicle maintenance date
[6] <licence plate>] Do maintenance
>>> 1
> TCP Server initialized - listening on port 1337
> Waiting for RSU connections...

--- [ List of registered vehicles ] ---

> 66-66-RR L0: 2020-06-16 16:09:29, LM: never, NM: not scheduled
>>> |
```

Figura 4.1 – Lista de veículos registados pela marca.

O registo de veículos consiste no envio do endereço IPv6 (apenas os campos pertencentes à máscara definida, sendo posteriormente concatenada à parte do endereço pertencente à gama, tornando-se num método de validação e facilitismo), da matrícula e opcionalmente do país, ano e mês para o IMTT, para que seja atribuído um endereço DNS ao veículo, seja registado na base de dados do IMTT e da marca e para criar um certificado assinado pela marca e uma *keystore* com a chave privada do veículo.

O registo e pedido do mesmo ao IMTT pela marca Buffalo está demonstrado na Figura 4.2.

```
root@n2: /home/edsantos/PTI2/network/n2
[2] List of active IMTT alerts
[3] List of all events
[4 <address> <licence plate> (<country> <year> <month>)] Register vehicle
[5 <licence plate> <yyyy-MM-dd>] Schedule vehicle maintenance date
[6 <licence plate>] Do maintenance
>>> 1
> TCP Server initialized - listening on port 1337
> Waiting for RSU connections...

--- [ List of registered vehicles ] ---
> 66-66-RR L0: 2020-06-16 16:09:29, LM: never, NM: not scheduled

>>> 4 200;ff:feaa:d 13-37-PT
> Registering P 13-37-PT
>>> 1
> Vehicle DNS: 13-37-PT, 1337pt.auto.pt

--- [ List of registered vehicles ] ---
> 66-66-RR L0: 2020-06-16 16:09:29, LM: never, NM: not scheduled
> 13-37-PT L0: never, LM: never, NM: not scheduled

>>> |
```

Figura 4.2 – Registo de veículo feito pela marca com atribuição de DNS pelo IMTT.

4.1.2 Alertas

Sempre que é recebido um alerta pelo IMTT, é inserido (ou atualizado) na tabela **Alert**, constituída pelas seguintes colunas:

- **id** (chave primária) – identificador numérico do alerta;
- **tstamp** – *timestamp* da inserção do alerta;
- **ttl** – *timestamp* da validade do alerta;
- **maxTtl** – TTL máximo atribuído ao alerta;
- **latitude** – latitude (em graus) do epicentro do alerta;
- **longitude** – longitude (em graus) do epicentro do alerta;
- **radius** – raio de alcance (em metros) do alerta a partir do seu epicentro;
- **message** – descrição do alerta.

Esta tabela é percorrida a cada 10 segundos e todos os alertas que não tenham o **maxTtl** ultrapassado são encaminhados para os RSU's. O **maxTtl** é importante pois se for usado o campo **ttl** e este for inferior ao **maxTtl**, os OBU's ao terem um alerta armazenado com um certo TTL verificarão a validade desse alerta e não o irão alterar se não for recebida uma mensagem com um novo TTL. Por exemplo, se um OBU tiver recebido um alerta com validade para as 18h e se a marca receber uma atualização desse alerta para cancelar o alerta (ao cancelar, o TTL torna-se na *timestamp* atual, portanto suponhamos que seja 16h), a marca deixará de enviar o alerta com o novo TTL (16h) pois o mesmo foi ultrapassado pela *timestamp* atual (16h) e o OBU nunca irá saber que o alerta foi cancelado e permanecerá com o TTL das 18h.

Na Figura 4.3 encontra-se o diagrama de sequência deste processo.

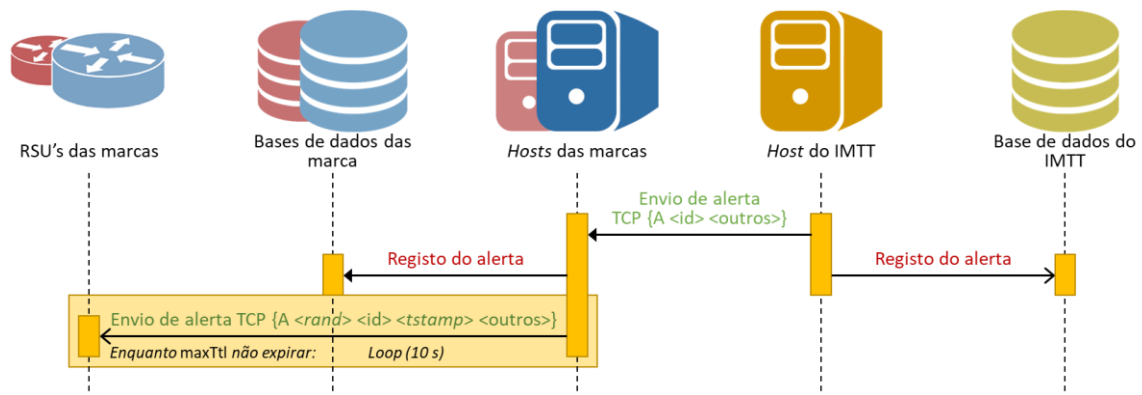


Figura 4.3 – Diagrama de sequência do lançamento de um alerta pelo IMTT.

Na Figura 4.4 encontra-se a receção e a listagem de um alerta “More traffic” recebido pela marca Buffalo.

```

root@n2: /home/edsantos/PTI2/network/n2

--- [ List of registered vehicles ] ---
> 66-66-RR L0: 2020-06-16 16:09:29, LM: never, NM: not scheduled
>>> 4 200;ff:feaa:d 13-37-PT
> Registering P 13-37-PT
>>> 1
> Vehicle DNS: 13-37-PT, 1337pt.auto.pt

--- [ List of registered vehicles ] ---
> 66-66-RR L0: 2020-06-16 16:09:29, LM: never, NM: not scheduled
> 13-37-PT L0: never, LM: never, NM: not scheduled

>>> 2
> IMTT alert: #105 More traffic until 2020/06/22 15:32:30

--- [ List of active IMTT alerts ] ---
> #105 [2020-06-22 15:31:33] More traffic at (0.00, 0.00) ±0 m until 2020-06-22 15:32:30
>>>

```

Figura 4.4 – Alerta do IMTT recebido e listado pela marca.

4.1.3 Eventos

A tabela **Event** tem o propósito de servir como um *logbook*, contendo o histórico de alertas IMTT (inserção e atualização), registo de veículos, atribuição de DNS a veículos, mensagens de erro e mensagens correspondentes a serviços (registo de manutenção, agendamento de manutenção, alerta de ultrapassagem da data de manutenção, depósito de combustível vazio, pedidos de SOS e desativação de pedidos de SOS). As colunas desta tabela são as seguintes:

- **id** (chave primária, autoincrementável) – identificador numérico do evento;
- **tstamp** – *timestamp* da ocorrência do evento;
- **type** – tipo de evento;
- **source** – origem do evento;
- **destination** – destino do evento;
- **ttl** – *timestamp* de validade do evento;
- **latitude, longitude** – coordenadas do epicentro do evento;
- **radius** – raio de alcance do evento;
- **message** – descrição do evento.

Nos eventos não específicos a serviços exclusivos de marca, os campos são preenchidos de acordo com a Tabela 4.1.

Tabela 4.1 – Estrutura dos eventos gerais armazenados na tabela de eventos.

Evento	type	source	destination	ttl	latitude, longitude	radius	message
Alerta IMTT	"A" + id do alerta	"imtt"	"all Vehicles"	TTL do alerta	coordenadas do alerta	alcance do alerta	descrição do alerta
Registo de veículo	"V"	nome da marca	"imtt"	-			endereço IPv6, matrícula, país, ano e mês
Atribuição de DNS	"D"	"imtt"	nome da marca	-			endereço DNS e matrícula
Erro	"E"	"imtt" ou nome da marca		-			descrição do erro

Um exemplo de listagem de alguns eventos encontra-se na Figura 4.5, tendo o registo de um veículo, de uma atribuição de DNS e de dois alertas IMTT.

```
root@n2: /home/edsantos/PTI2/network/n2
- TTL: 2020-06-22 11:30:01
- message: Traffic

> #136 [2020-06-22 15:26:31]
- type: Vehicle registration
- source: Buffalo
- destination: IMTT
- message: 200:ff:feaa:d, 13-37-PT, P, 00, 00

> #137 [2020-06-22 15:26:32]
- type: Vehicle DNS
- source: IMTT
- destination: Buffalo
- message: 13-37-PT, 1337pt.auto.pt

> #138 [2020-06-22 15:31:32]
- type: Alert #105
- source: IMTT
- destination: all vehicles
- TTL: 2020-06-22 15:32:30
- message: More traffic

>>> █
```

Figura 4.5 – Lista de eventos armazenados.

4.2 Certificados e encriptação

Para garantir a confidencialidade das mensagens trocadas cujo destino é para uma entidade (veículo ou marca) em específico, estas são encriptadas usando encriptação assimétrica. Este tipo de encriptação permite que haja uma chave de descriptação para cada utilizador, não permitindo que a mensagem destinada a este não possa ser descriptada por mais ninguém.

A chave de descriptação é uma chave privada, só conhecida pelo seu dono e é usada apenas para descriptar mensagens destinadas ao mesmo. Esta chave tem 256 bytes, faz parte de um par de chaves RSA (*Rivest–Shamir–Adleman*) e encontra-se armazenada numa *keystore*.

Também pertencente a este par de chaves, encontra-se uma chave pública gerada a partir da chave privada que como o nome indica, pode e deve ser conhecida por outras entidades, principalmente as que pretendem enviar uma mensagem para o dono dessa chave pois será usada para encriptar tais mensagens.

A troca de chaves públicas é feita através de certificados para garantir a autenticidade das mesmas. Se a chave pública não acompanhasse o seu certificado, nada garantia que a chave pública pertence a quem a enviou, e se não pertencesse, as mensagens encriptadas com essa chave só seriam descriptadas por um dono inesperado. Como os certificados são assinados por uma entidade certificadora que é a marca, confiada pelos veículos da mesma, e como todos os veículos têm a posse do certificado dessa entidade (auto-assinado), podem verificar a assinatura dos certificados recebidos, pois foi essa CA (*Certificate Authority*) que assinou os certificados legítimos dos veículos.

Na maior parte dos serviços (e em todos da marca Cheetah), as mensagens têm a marca como destino ou origem, portanto todos os veículos não só contêm o seu certificado como também o certificado da marca. As marcas também contêm não só o seu certificado (auto-assinado) como também os certificados de todos os veículos, gerados no registo dos mesmos, como se pode ver na Figura 4.6.

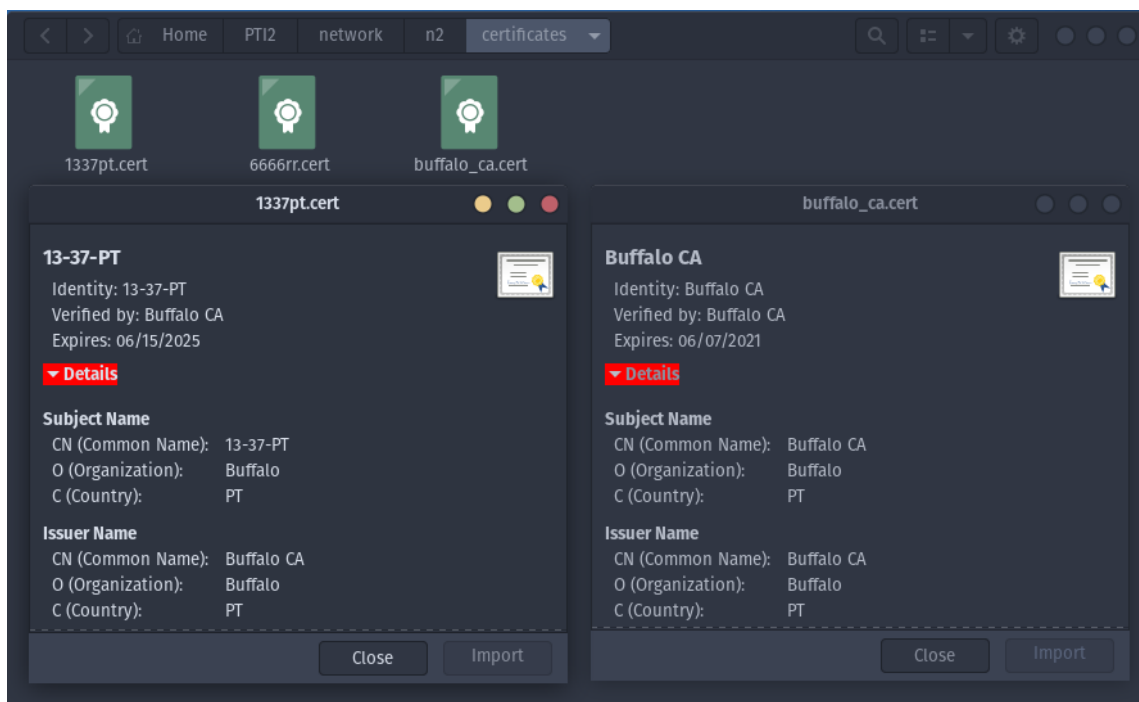


Figura 4.6 – Certificados armazenados no *host* da marca.

Nas mensagens enviadas, apenas o corpo e a origem das mesmas se encontram encriptados, pois é necessário saber previamente se se refere a um serviço e não um alerta (identificado no cabeçalho) e qual o destinatário para se saber se se deve proceder à descriptação do corpo da mensagem.

4.2.1 Registo de veículos

Como o certificado e a chave privada dum OBU são elementos necessário da configuração da sua aplicação veicular, é compreensível que os mesmos não tenham de ser enviados da marca para o OBU a partir da rede veicular pois o OBU necessita desses ficheiros para participar nessa rede. Como tal, os certificados e *keystores* são colocados na diretoria do OBU, demonstrado na Figura 4.7, após o respetivo veículo ser registado, sendo a criação destes ficheiros parte desse registo.

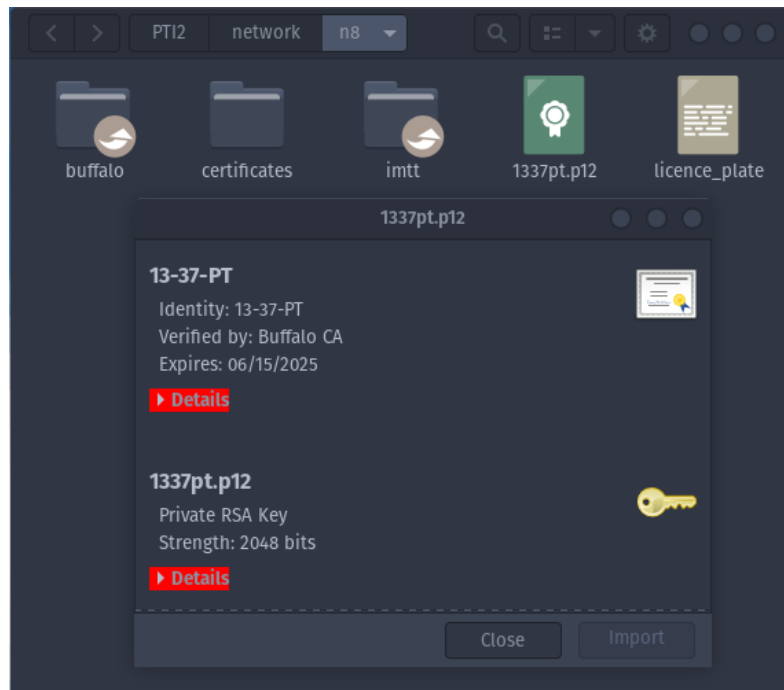


Figura 4.7 – Keystore armazenada por um OBU.

No diagrama de sequência da Figura 4.8 está demonstrado o processo de registo de um veículo.

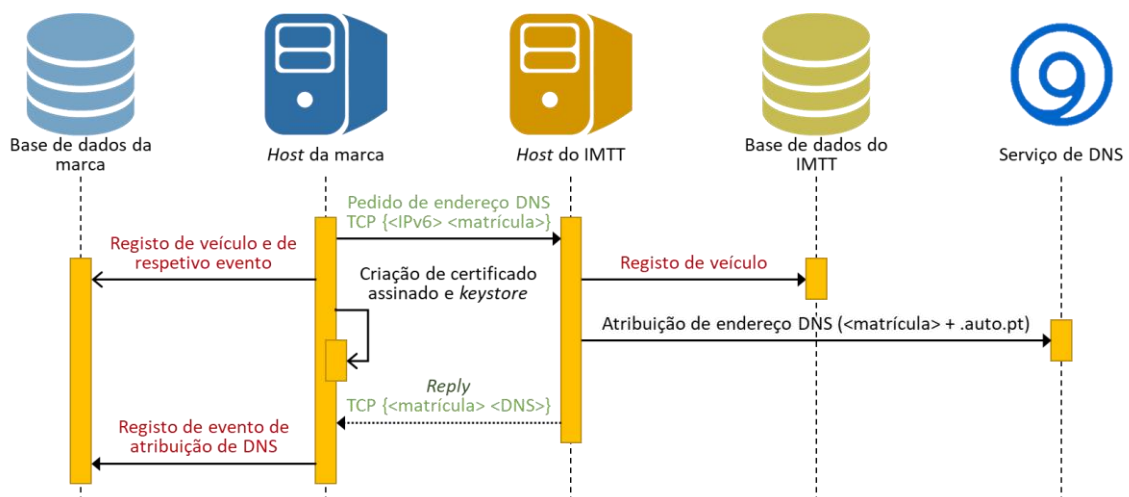


Figura 4.8 – Diagrama de sequência do registo de um veículo.

Nota: Para a criação de certificados, chaves privadas e *keystores*, foram reciclados *script* em Python construídos pelos mesmos elementos do grupo na UC (Unidade Curricular) de Criptografia no semestre passado.

4.3 Protocolo de comunicação

O protocolo de comunicação da rede é maioritariamente orientado ao carácter, sendo cada parâmetro do cabeçalho separado com um carácter de espaço.

4.3.1 Registo de veículos

Para o registo de veículos, a marca envia para o IMTT mensagens com a seguinte estrutura:

V <address> <licencePlate> <country> <year> <month>

- **address** – campos pertencentes à máscara definida do endereço IPv6 do OBU, sendo posteriormente concatenados pelo IMTT à parte do endereço pertencente à gama (*:821:);
- **licencePlate** – matrícula do veículo;
Opcional:
- **country** – carácter identificador do país;
- **year** – ano de registo legal da matrícula;
- **month** – mês (numérico) de registo legal da matrícula.

Após o registo de um veículo e da sua atribuição de endereço DNS, o IMTT envia uma mensagem de volta à marca com a seguinte estrutura:

D <licencePlate>, <dnsAddress>

- **dnsAddress** – endereço DNS do veículo (*.auto.pt).

4.3.2 Alertas

Quando um alerta é emitido, atualizado ou cancelado pelo IMTT, é enviada para a marca a seguinte estrutura de mensagem:

A <id> <ttl> <latitude> <longitude> <radius> <message>

- **id** (chave primária) – identificador numérico do alerta;
- **ttl** – *Unix timestamp* (em segundos) da validade do alerta;
- **latitude** – latitude (em graus) do epicentro do alerta;
- **longitude** – longitude (em graus) do epicentro do alerta;
- **radius** – raio de alcance (em metros) do alerta a partir do seu epicentro;
- **message** – descrição do alerta.

Na receção duma mensagem de alerta pela marca e pelo reenvio dos alertas armazenados não expirados a cada 10 segundos, é enviada uma mensagem para os RSU's com a seguinte estrutura:

A *<rand>* *<id>* *<tstamp>* *<ttl>* *<latitude>* *<longitude>* *<radius>* *<message>*

- **rand** – número aleatório de 16 bits ($\in [0, 65535]$), para garantir a unicidade da mensagem na existência de outro alerta com os mesmos parâmetros;
- **tstamp** – *Unix timestamp* (em segundos) do encaminhamento do alerta pela marca; útil para um OBU saber se a mensagem de alerta recebida está atualizada (com *timestamp* mais recente do que o alerta armazenado, no caso de atualização de um existente).

4.3.3 Mensagens de serviços

Os restantes tipos de mensagens têm campos exclusivos aos serviços no seu corpo, contudo a maior parte tem uma estrutura semelhante de cabeçalho:

S *<rand>* *<brandId>* *<destination>* *<type>* (*<sourcePlate>*) *<body>*

- **brandId** – identificador numérico da marca;
- **destination** – “brand” se o destino for a marca, ou matrícula do veículo se o destino for um OBU;
- **type** – caracter identificador do serviço;

Encriptado:

- **sourcePlate** – matrícula do veículo se a origem for um OBU;
- **body** – corpo da mensagem, dependente do tipo de serviço.

Assim como as mensagens de alerta, também contêm o campo **rand** para garantir a unicidade da mensagem na existência de outra com os mesmos parâmetros. Esta garantia é importante pois mensagens repetidas não devem ser reencaminhadas por um OBU ou RSU pois originaria num ciclo infinito de trocas de mensagens em *multicast* em veículos no alcance um do outro pois quando um veículo *A* envia uma mensagem em *multicast*, *B* recebe-a e reencaminha-a novamente em *multicast*, fazendo com que *A* a receba e volte a repetir o processo infinitamente, até estarem fora do alcance um do outro.

Para garantir que não há mensagens repetidas, é necessário armazenar as mensagens recebidas para comparar com as próximas que forem recebidas, de forma a apenas serem processadas e reencaminhadas (caso o recetor não seja o destino exclusivo das mesmas) mensagens que não estejam previamente armazenadas. Estas mensagens são armazenadas numa espécie de *cache* que contém uma lista de mensagens com um número máximo de 256 entradas. Ao ser atingido este número, a mensagem mais antiga é removida.

Este processo encontra-se demonstrado no fluxograma da Figura 4.9.

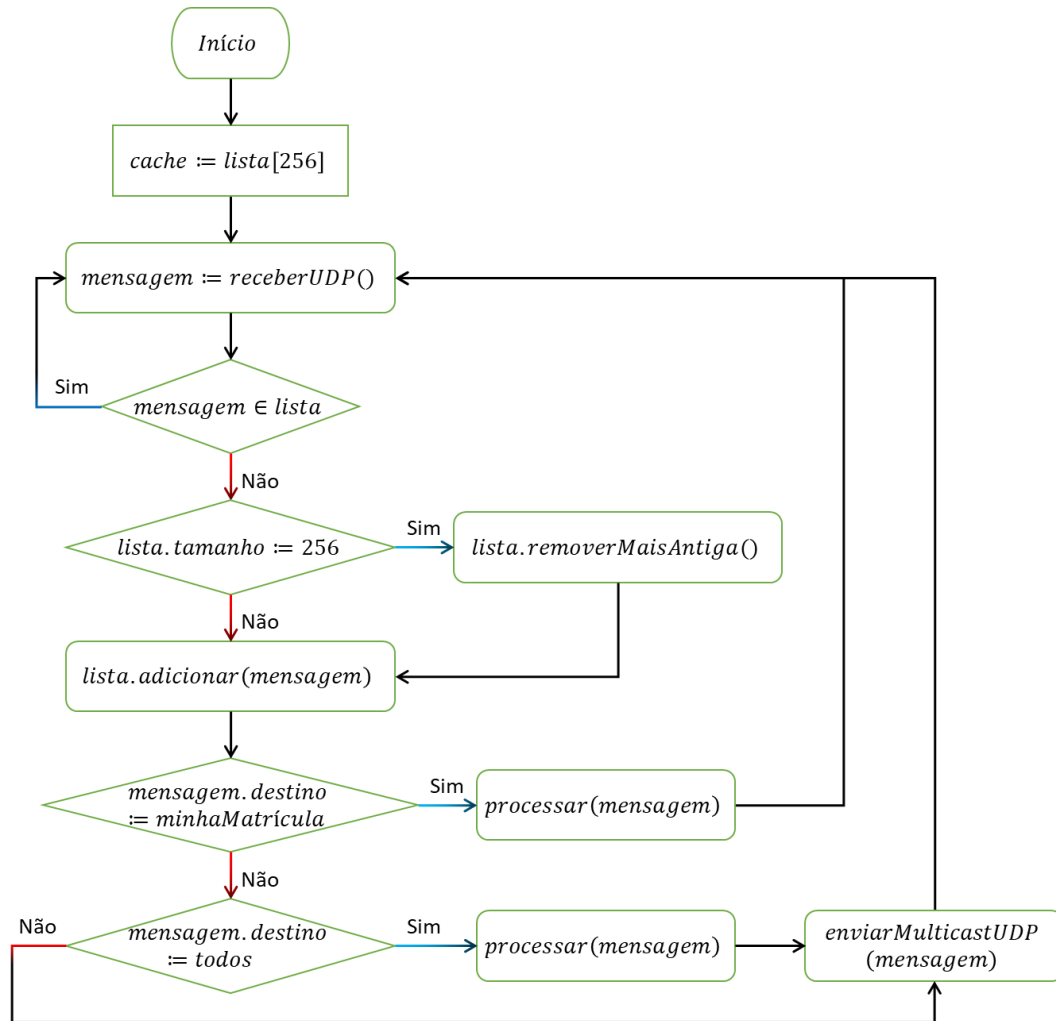


Figura 4.9 – Fluxograma da recepção de mensagens pelos OBU's.

Nota: Temos a noção de que esta estrutura de mensagens não é a mais ótima por ser maioritariamente orientada ao carácter, sendo a estrutura ideal orientar ao bit (ocupar por exemplo o valor 10 000 em 2 bytes em vez de 5 (um por algarismo/carácter) e reservar um tamanho fixo de bytes a cada campo em vez de separá-los por um espaço, que por sua vez ocupa 1 byte), contudo a opção escolhida foi mais simples de implementar mas se tivéssemos mais tempo, seria seguida a alternativa referida; apesar de as desvantagens do sistema atual não serem notáveis devido ao tamanho reduzido de cada pacote.

4.4 RSU's

O *host* de uma marca trata-se de um servidor TCP *multi-thread* para os RSU's, e à semelhança do *host* IMTT, é criada uma *thread* para cada conexão de um RSU, de modo a haver comunicação assíncrona com todos os RSU's. O RSU dispõe de um programa em Java para executar as suas funções.

O papel do RSU é de encaminhar as mensagens recebidas pela marca para a rede veicular em *multicast*, assim como vice-versa caso as mensagens sejam destinadas à marca.

As mensagens da marca são recebidas por TCP e são convertidas em UDP para serem enviadas em *multicast*, no sentido oposto, as mensagens recebidas da rede veicular são em UDP e se forem destinadas para a marca, são enviadas em TCP para a mesma.

Na Figura 4.10 encontra-se o diagrama de sequência do encaminhamento das mensagens originadas na marca, destinadas a um ou vários veículos.

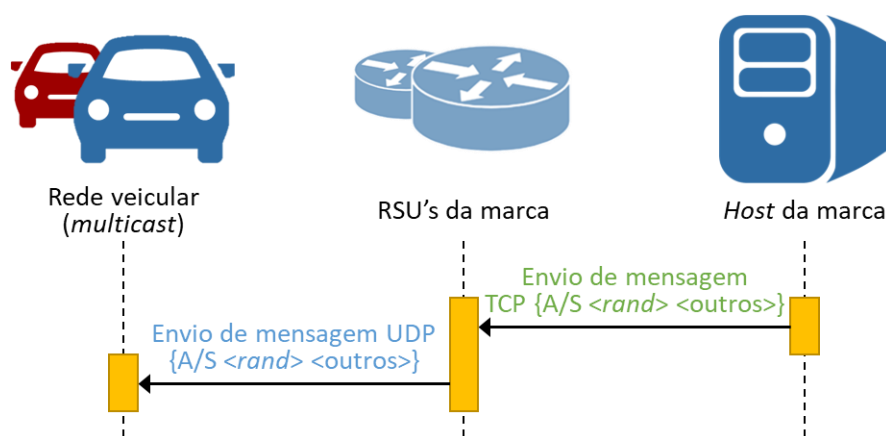


Figura 4.10 – Diagrama de sequência do envio de mensagens do *host* da marca para a rede veicular via RSU's.

Apenas mensagens recebidas pelo RSU com identificador numérico da sua marca (X) e destinadas à marca ("brand") são enviadas para o *host*, demonstrado na Figura 4.11.

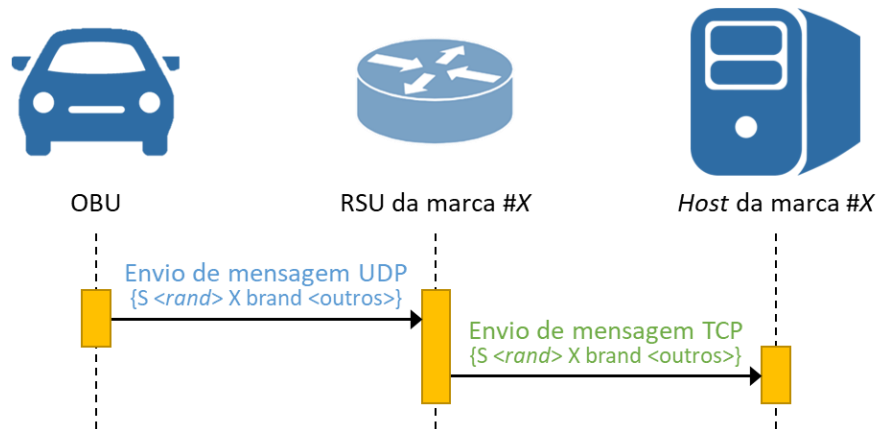


Figura 4.11 – Diagrama de sequência da recepção de mensagens do *host* da marca enviadas pela rede veicular via RSU's.

Quando as condições estabelecidas não são estabelecidas, a mensagem é devolvida à rede veicular, como se pode ver na Figura 4.12 onde a mensagem é destinada a um veículo, e na Figura 4.13 onde a mensagem é dum serviço de outra marca (Y).

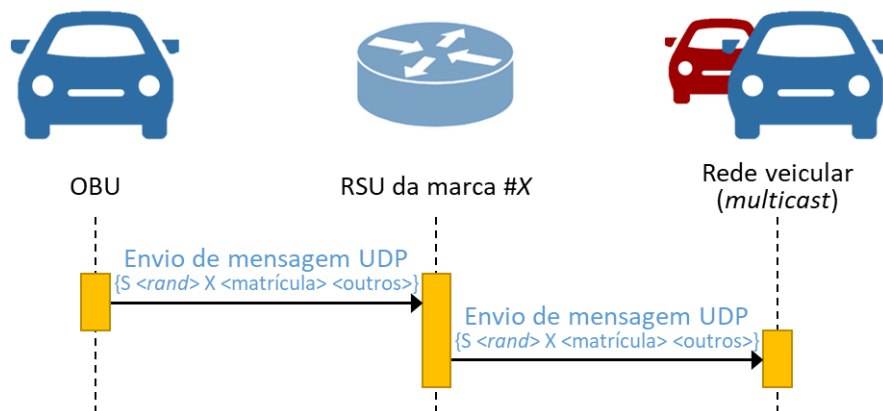


Figura 4.12 – Diagrama de sequência da devolução de mensagens destinadas a veículos para a rede veicular.

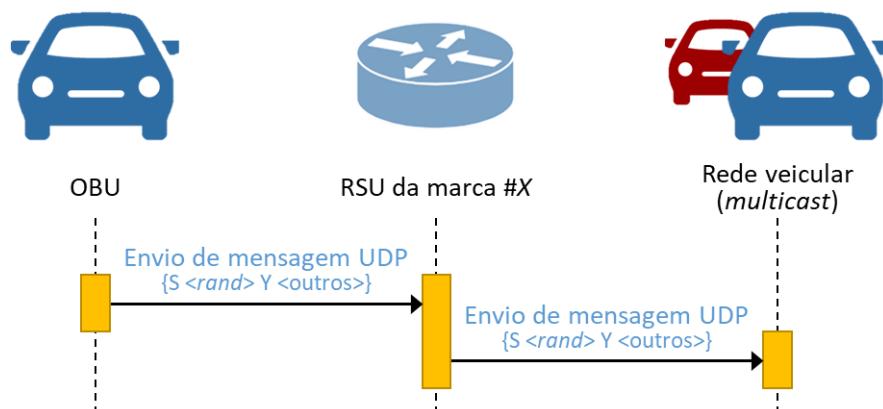
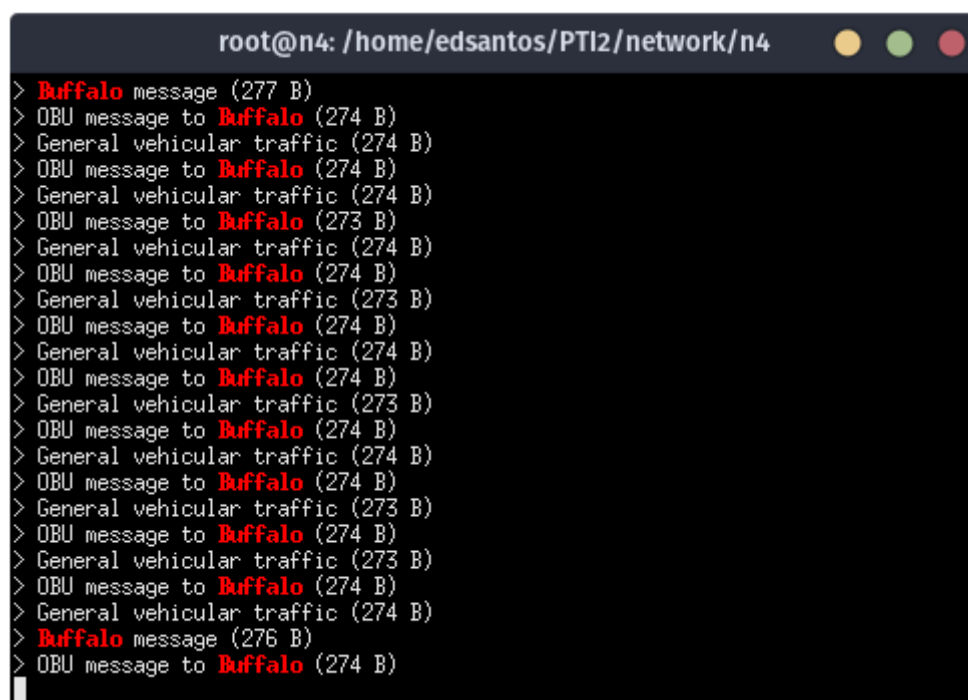


Figura 4.13 – Diagrama de sequência da devolução de mensagens de outras marcas para a rede veicular.

Mensagens de alerta originados do IMTT são obviamente devolvidos à rede veicular.

Na Figura 4.14 encontra-se a sinalização de todas as mensagens passadas por RSU, incluindo mensagens da marca (“Buffalo message”), para a marca (“OBU message to Buffalo”) e outras (outras marcas, direcionadas de OBU’s para OBU’s ou repetidas) (“General vehicular traffic”).

A terminal window with a dark background and light text. The title bar at the top reads 'root@n4: /home/edsantos/PTI2/network/n4'. The terminal content shows a list of messages, each preceded by a red prompt character '>'. The messages are: 'Buffalo message (277 B)', 'OBU message to Buffalo (274 B)', 'General vehicular traffic (274 B)', 'OBU message to Buffalo (274 B)', 'General vehicular traffic (274 B)', 'OBU message to Buffalo (273 B)', 'General vehicular traffic (274 B)', 'OBU message to Buffalo (274 B)', 'General vehicular traffic (273 B)', 'OBU message to Buffalo (274 B)', 'General vehicular traffic (274 B)', 'OBU message to Buffalo (274 B)', 'General vehicular traffic (273 B)', 'OBU message to Buffalo (274 B)', 'General vehicular traffic (274 B)', 'OBU message to Buffalo (274 B)', 'General vehicular traffic (273 B)', 'OBU message to Buffalo (274 B)', 'General vehicular traffic (274 B)', 'OBU message to Buffalo (274 B)', 'General vehicular traffic (273 B)', 'OBU message to Buffalo (274 B)', 'General vehicular traffic (274 B)', 'Buffalo message (276 B)', and 'OBU message to Buffalo (274 B)'.

```
root@n4: /home/edsantos/PTI2/network/n4
> Buffalo message (277 B)
> OBU message to Buffalo (274 B)
> General vehicular traffic (274 B)
> OBU message to Buffalo (274 B)
> General vehicular traffic (274 B)
> OBU message to Buffalo (273 B)
> General vehicular traffic (274 B)
> OBU message to Buffalo (274 B)
> General vehicular traffic (273 B)
> OBU message to Buffalo (274 B)
> General vehicular traffic (274 B)
> OBU message to Buffalo (274 B)
> General vehicular traffic (273 B)
> OBU message to Buffalo (274 B)
> General vehicular traffic (274 B)
> OBU message to Buffalo (274 B)
> General vehicular traffic (273 B)
> OBU message to Buffalo (274 B)
> General vehicular traffic (274 B)
> OBU message to Buffalo (274 B)
> General vehicular traffic (273 B)
> OBU message to Buffalo (274 B)
> General vehicular traffic (274 B)
> Buffalo message (276 B)
> OBU message to Buffalo (274 B)
```

Figura 4.14 – Sinalização das mensagens passadas por um RSU.

4.5 OBU’s – Veículos

Todos os OBU’s dos veículos dispõem de uma aplicação veicular em Java, uma pasta de certificados com o seu certificado, CA e eventualmente de outros veículos, *keystore* com a sua chave privada e um ficheiro a definir a matrícula do veículo.

Toda a comunicação da rede veicular é feita a partir de UDP, sendo todas as mensagens enviadas em *multicast*.

Todas as mensagens recebidas são reencaminhadas em *multicast* se não tiverem previamente armazenadas em *cache* e se o destinatário não for exclusivamente quem a recebeu.

Para além do encaminhamento e armazenamento de mensagens, a aplicação veicular do OBU dispõe de diversas outras funcionalidades.

4.5.1 Serviço de localização geográfica e de estado *online*

Através da API (*Application Programming Interface*) gRPC (*Google's Remote Procedure Calls*) disponibilizada pelo CORE (versão > 5.1) [21], foi desenvolvido um *script* em Python que a partir do número identificador da sessão da topologia, consegue obter as coordenadas XY de um dado nó e convertê-las para latitude e longitude. Este *script* corre em *background* no computador que hospeda a topologia e espera por mensagens UDP que contenham o identificador dum nó, para responder com as suas coordenadas.

A cada segundo, cada veículo envia o seu identificador do nó numa mensagem UDP ao endereço do computador *host* e espera até 5 segundos por uma resposta com as suas coordenadas. Ao ser recebida uma mensagem ou atingido o *timeout*, é enviado para a rede veicular com destino à marca uma espécie de *keep alive* para determinar há quanto tempo é que o veículo se encontra *online*, e dependendo da marca, com as suas coordenadas incluídas (no caso de *timeout* são enviadas as últimas coordenadas obtidas). Esta mensagem tem a seguinte estrutura:

$S \langle rand \rangle \langle brandId \rangle brand\ O \langle licencePlate \rangle (\langle body \rangle)$

Encriptado:

- **licencePlate** – matrícula de quem emitiu a mensagem;
- **body** – corpo da mensagem – dependendo da marca, pode conter as coordenadas do veículo e outros parâmetros.

Na Figura 4.15 encontra-se o processo deste serviço.

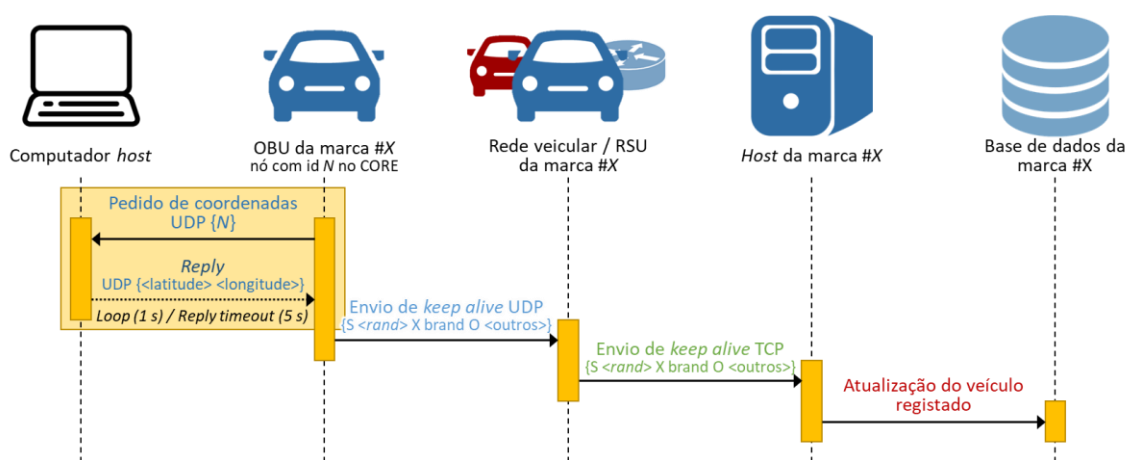


Figura 4.15 – Diagrama de sequência da obtenção de coordenadas GPS e envio de *keep alive*.

Logicamente, num sistema real, as coordenadas seriam obtidas via GPS (*Global Positioning System*).

Na Figura 4.16 encontra-se um *snapshot* da aplicação veicular Buffalo com as coordenadas de um veículo.

```
root@n8: /home/edsantos/PTI2/network/n8

=== [ Buffalo OBU 13-37-PT ] ===

--- [ Main menu ] ---

[1] List of active alerts
[2] <vehicle plate> Call user
[3] <vehicle plate> Share location with user
[4] Do maintenance
[5] List of stored certificates
>>>

[Location] 41.4480545828748759, -8.2964715956919899
[Scheduled maintenance] Not needed for a while
[IMTT alerts] #107 Can there be too much traffic? until 2020/06/22 16:08:53 [+1]
[Voice call] Microphone not in use
[Voice call] Not receiving audio
[Location sharing] Disabled
```

Figura 4.16 – Coordenadas GPS e alerta ativo mais recente visíveis na aplicação veicular.

4.5.2 Alertas

Quando um alerta é recebido, se não houver um existente com o mesmo ID ou caso haja, se a *timestamp* contida for mais recente do que a do alerta existente, é armazenado e, no caso de existência prévia, atualizado. Se for recebido um alerta com TTL expirado, este é ignorado, e caso esteja previamente armazenado, é removido da lista. Quando é recebido um alerta com TTL ainda em vigor, este é também reencaminhado para a rede veicular.

A cada 3 segundos, a lista armazenada de alertas é reanalisada, para verificar se há alertas expirados e alertas no alcance de notificação. Na existência de alertas expirados, estes são removidos.

Um alerta é considerado no alcance de notificação / ativo quando os parâmetros de latitude, longitude e raio têm valor = 0 (referido anteriormente) ou quando:

$$d \geq 2 \times r$$

- d = distância do veículo ao epicentro do alerta;
- r = raio de alcance do alerta.

É determinado que um veículo se encontra no local do alerta quando $d \leq r$, caso contrário, é-lhe revelado a distância $d - r$, de acordo com a Figura 4.17.

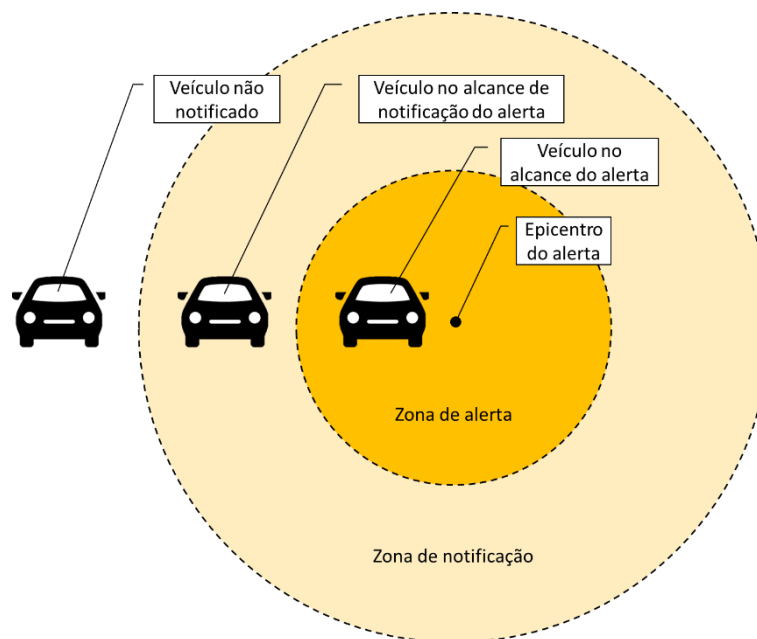


Figura 4.17 – Esquema das zonas de alerta e de notificação.

No menu principal da aplicação veicular, é apresentado o alerta ativo mais recente (descrição e distância em metros), juntamente com o número adicional de alertas ativos existentes, visto anteriormente na Figura 4.16. O utilizador pode também visualizar a lista de todos os alertas ativos, vista na Figura 4.18.

```

root@n8: /home/edsantos/PTI2/network/n8

=== [ Buffalo OBU 13-37-PT ] ===

--- [ List of active IMT alerts ] ---

> #106 [2020/06/22 16:08:08] Even more traffic at (0.02, 0.02) until 2020/06/22
16:08:31
> #107 [2020/06/22 16:07:54] Can there be too much traffic? at (0.02, 0.02) unti
l 2020/06/22 16:08:53

[0] Go back
[1] Refresh list
>>>

```

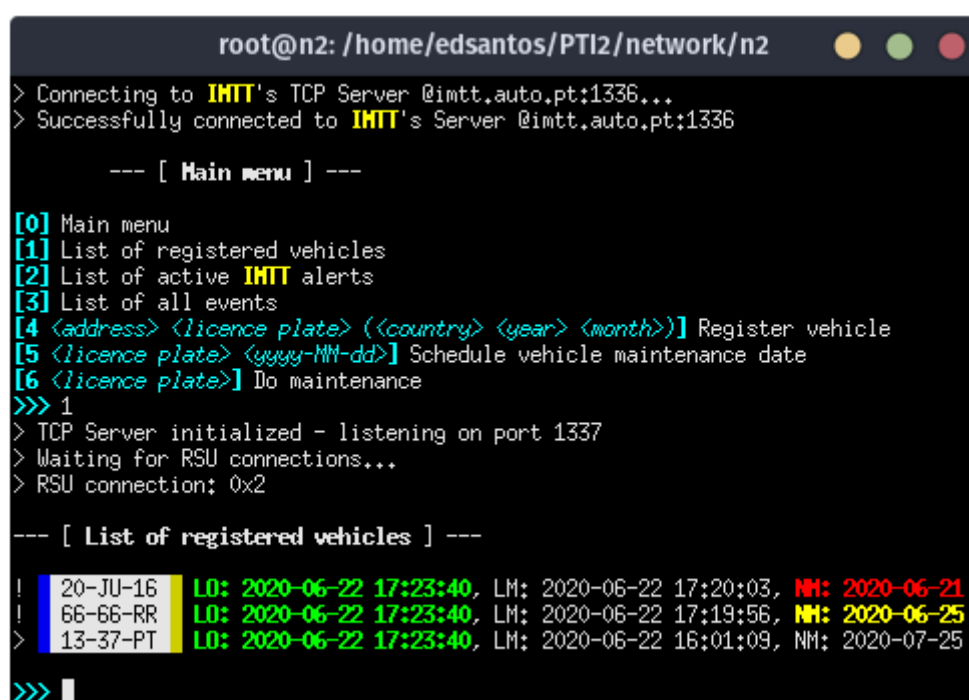
Figura 4.18 – Lista de alertas ativos no OBU.

5 Marca Buffalo #1

A marca Buffalo é uma marca notável pelos seus serviços de comunicação entre veículos. Os serviços desta marca são: alertas de manutenção, partilha de localização entre veículos e chamadas de voz entre veículos.

5.1 Monitorização de veículos

A monitorização de veículos é feita pelo *host* da marca, demonstrada na Figura 5.1, em que a listagem dos mesmos contém a respetiva matrícula, *timestamp* da última mensagem de *keep alive* recebida (**LO** – *Last time Online*), *timestamp* da última manutenção feita (**LM** – *Last Maintenance*) e data da próxima manutenção agendada (**NM** – *Next Maintenance*).



```
root@n2: /home/edsantos/PTI2/network/n2
> Connecting to IMTT's TCP Server @imtt.auto.pt:1336...
> Successfully connected to IMTT's Server @imtt.auto.pt:1336

--- [ Main menu ] ---
[0] Main menu
[1] List of registered vehicles
[2] List of active IMTT alerts
[3] List of all events
[4] <address> <licence plate> <<country> <year> <month>>> Register vehicle
[5] <licence plate> <yyy-MM-dd> Schedule vehicle maintenance date
[6] <licence plate> Do maintenance
>>> 1
> TCP Server initialized - listening on port 1337
> Waiting for RSU connections...
> RSU connection: 0x2

--- [ List of registered vehicles ] ---
! 20-JU-16 LO: 2020-06-22 17:23:40, LM: 2020-06-22 17:20:03, NM: 2020-06-21
! 66-66-RR LO: 2020-06-22 17:23:40, LM: 2020-06-22 17:19:56, NM: 2020-06-25
> 13-37-PT LO: 2020-06-22 17:23:40, LM: 2020-06-22 16:01:09, NM: 2020-07-25
>>> 
```

Figura 5.1 – Monitorização de veículos da marca Buffalo.

Nesta lista são realçados diversos estados:

- **Veículo online:** LO de há menos de 60 segundos;
- **Alerta de manutenção:** NM nos próximos 7 dias;
- **Manutenção ultrapassada:** NM ultrapassada.

Nesta marca, a mensagem de *keep alive* não contém nenhum corpo, sendo o cabeçalho o seguinte:

S <rand> 1 brand *O* <licencePlate>

Nota: na Figura 5.1, sabemos que no veículo “20-JU-16” devia ser impossível a última manutenção ser anterior à próxima, sendo este exemplo usado apenas para comparar a próxima manutenção (21 de junho) com a data atual (22 de junho).

5.2 Troca de certificados

Nos serviços de partilha e localização e de chamadas de voz, para um veículo *A* comunicar com *B*, ambos os OBU’s necessitam de ambas as chaves públicas para poderem encriptar as mensagens. Como num estado inicial todos os OBU’s apenas contêm na sua diretoria de certificados o seu próprio certificado e a CA da marca, é necessário obter o certificado pretendido.

O pedido de certificado é feito pelo veículo *A* quando este encriptar uma mensagem para *B* mas não tem o certificado de *B* armazenado, como por exemplo, antes de enviar ou aceitar um pedido de partilha de localização ou de chamada de voz. Esta mensagem tem a seguinte estrutura:

S $\langle rand \rangle$ 1 *allVehicles* *C* $\langle wantedPlate \rangle$ $\langle requesterPlate \rangle$ $\langle requesterCert \rangle$

- **wantedPlate** – matrícula cujo certificado pretende ser obtido;
- **requesterPlate** – matrícula do veículo que fez o pedido;
- **requesterCert** – certificado do veículo que fez o pedido.

Como se pode ver, o destinatário é “allVehicles”, ou seja, todos os veículos e não só **wantedPlate**. Nesta condição, todos os veículos que receberem esta mensagem, independentemente de possuírem o certificado pedido, podem armazenar o certificado incluindo na mensagem para que não necessitem de o pedir posteriormente. Caso o veículo que receber a mensagem contenha a matrícula **wantedPlate**, este envia uma nova mensagem de estrutura semelhante, desta vez contendo o seu certificado e sem nenhuma **wantedPlate**:

S $\langle rand \rangle$ 1 *allVehicles* *C* – $\langle requestedPlate \rangle$ $\langle requestedCert \rangle$

- **requestedPlate** – matrícula do certificado pedido;
- **requestedCert** – certificado pedido.

Este processo encontra-se demonstrado no diagrama de sequência da Figura 5.2.

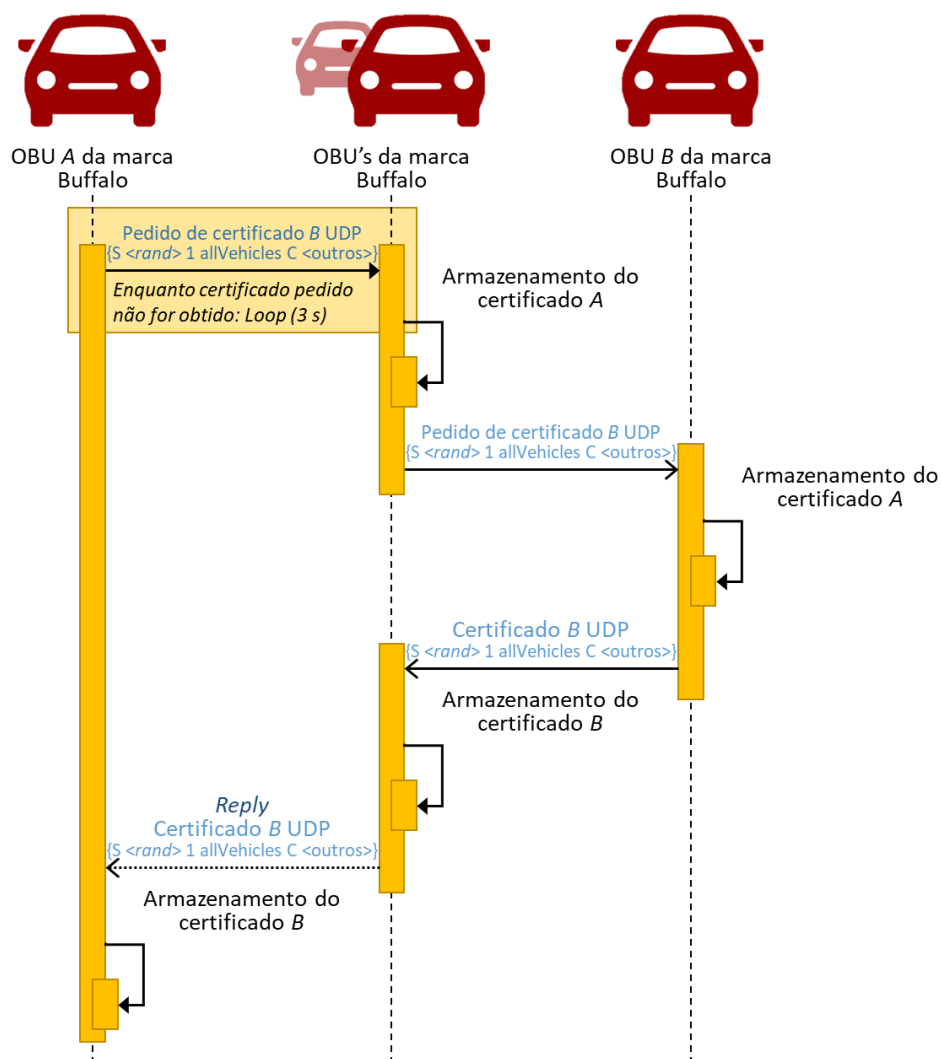


Figura 5.2 – Diagrama de sequência do pedido, troca e armazenamento de certificados.

O armazenamento de certificados só é feito na ausência dos mesmos.

Na Figura 5.3 encontra-se a troca de certificados feita entre n8 e n12, em que ambos adquiriram ambos os certificados. Como n9 se encontrava no meio da comunicação, conseguiu armazenar os dois certificados (n8 e n12) que por ele passaram.

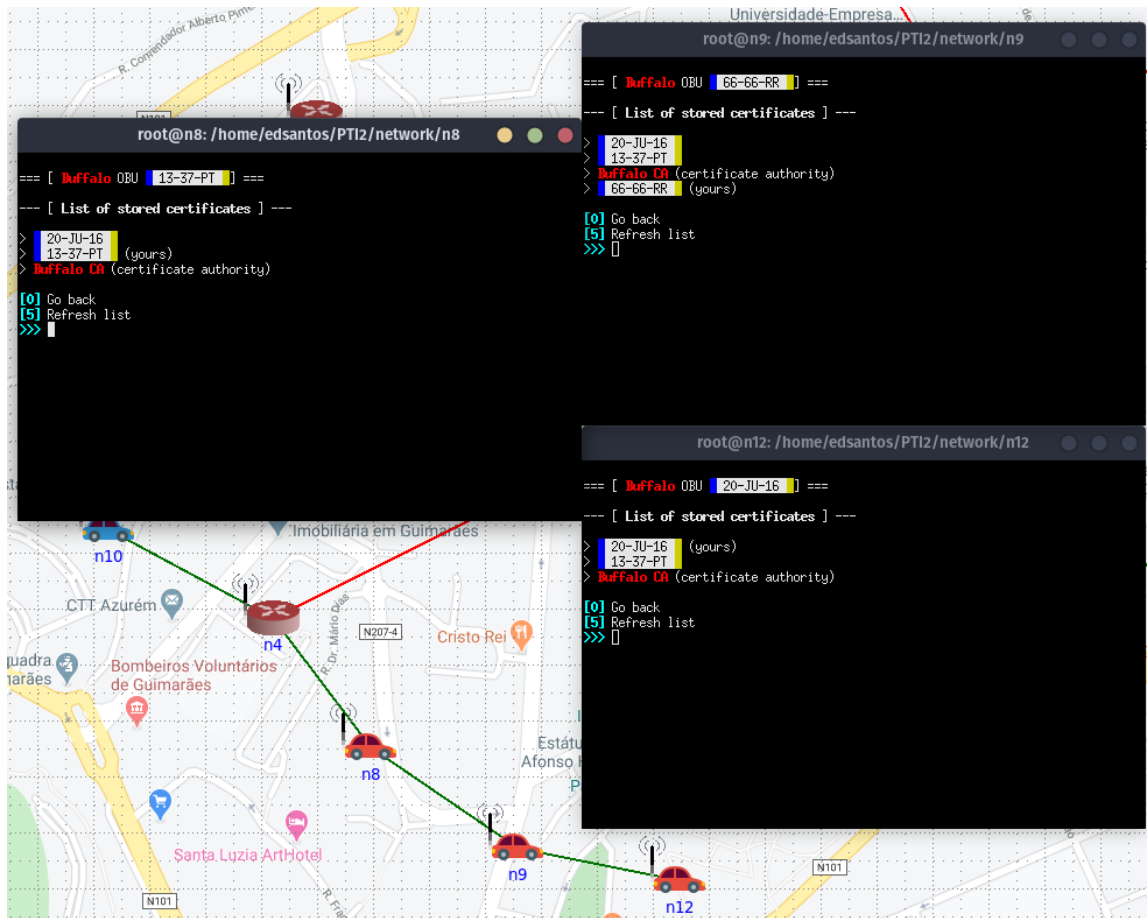


Figura 5.3 – Lista de certificados armazenados por três OBU's.

5.3 Serviços

5.3.1 Alertas de manutenção

O serviço de alertas de manutenção consiste em registrar, agendar e alertar datas de manutenção para veículos armazenados.

Para registrar uma manutenção, é inserida no *host* da marca a matrícula do veículo pretendido, sendo **LM** atualizado para a *timestamp* do registo.

Para agendar uma manutenção, são inseridas a matrícula do veículo pretendido e a data do agendamento (**NM**) (a hora é sempre 23:59:59).

Na Figura 5.4 encontra-se o agendamento e registo da manutenção do veículo “66-66-RR”.

```

root@n2: /home/edsantos/PTI2/network/n2

[0] Main menu
[1] List of registered vehicles
[2] List of active IMT alerts
[3] List of all events
[4] <address> <licence plate> (<country> <year> <month>)] Register vehicle
[5] <licence plate> <yyyy-MM-dd>] Schedule vehicle maintenance date
[6] <licence plate>] Do maintenance
>>> 5 13-37-PT 2020-7-25
> TCP Server initialized - listening on port 1337
> Waiting for RSU connections...
> RSU connection: 0x2
> Scheduling 13-37-PT's maintenance to 2020-7-25
>>> 6 13-37-PT
> Registering 13-37-PT's maintenance
>>> 1

--- [ List of registered vehicles ] ---

! 20-JU-16 L0: 2020-06-22 17:32:33, LM: 2020-06-22 17:20:03, NM: 2020-06-21
! 66-66-RR L0: 2020-06-22 17:32:33, LM: 2020-06-22 17:19:56, NM: 2020-06-25
> 13-37-PT L0: 2020-06-22 17:32:33, LM: 2020-06-22 17:32:30, NM: 2020-07-25

>>> 

```

Figura 5.4 – Agendamento e registo de manutenção de um veículo.

Tanto o registo como o agendamento de manutenções são armazenados na tabela de eventos da base de dados da marca, com os parâmetros da Tabela 5.1.

Tabela 5.1 – Estrutura do registo e agendamento de manutenção armazenados na tabela de eventos.

Evento	type	source	destination	ttl	latitude, longitude	radius	message
Registo de manutenção	"M"	"brand"	matrícula do veículo			-	
Agendamento de manutenção	"S"	"brand"	matrícula do veículo		-		timestamp agendada

A cada 10 segundos, a lista de veículos armazenados é percorrida, sendo procurados aqueles que estejam ativos nos últimos 60 segundos e com **NM** alcançável dentro de 7 dias. Para os veículos encontrados, é enviada uma mensagem de alerta de manutenção com a seguinte estrutura:

S <rand> 1 <licencePlate> M <lastMaintenance> <nextMaintenance>

Encriptado:

- **lastMaintenance** – Unix timestamp da última manutenção registada;
- **nextMaintenance** – Unix timestamp da próxima manutenção agendada.

Se **NM** for ultrapassado pela data atual, tanto a marca como o OBU são notificados.

Na Figura 5.5 e Figura 5.6 encontram-se as notificações de dois OBU's para datas de manutenção próximas e ultrapassadas.

```
root@n9: /home/edsantos/PTI2/network/n9

=== [ Buffalo OBU 66-66-RR ] ===

--- [ Main menu ] ---

[1] List of active alerts
[2] <vehicle plate> Call user
[3] <vehicle plate> Share location with user
[4] Do maintenance
[5] List of stored certificates
>>>

[Location] 41.447185971360739, -8.2947737798050029
[Scheduled maintenance] 2020/06/25 (last maintenance: 2020/06/22 17:19:56)
[IMTT alerts] No active alerts
[Voice call] Microphone not in use
[Voice call] Not receiving audio
[Location sharing] Disabled
```

Figura 5.5 – Alerta no OBU de data de manutenção a aproximar-se.

```
root@n12: /home/edsantos/PTI2/network/n12

=== [ Buffalo OBU 20-JU-16 ] ===

--- [ Main menu ] ---

[1] List of active alerts
[2] <vehicle plate> Call user
[3] <vehicle plate> Share location with user
[4] Do maintenance
[5] List of stored certificates
>>>

[Location] 41.446903165102529, -8.2929007924376129
[Scheduled maintenance surpassed] 2020/06/21 (last maintenance: 2020/06/22 17:20:03)
[IMTT alerts] No active alerts
[Voice call] Microphone not in use
[Voice call] Not receiving audio
[Location sharing] Disabled
```

Figura 5.6 – Alerta no OBU de data de manutenção ultrapassada.

Este processo está demonstrado na Figura 5.7.

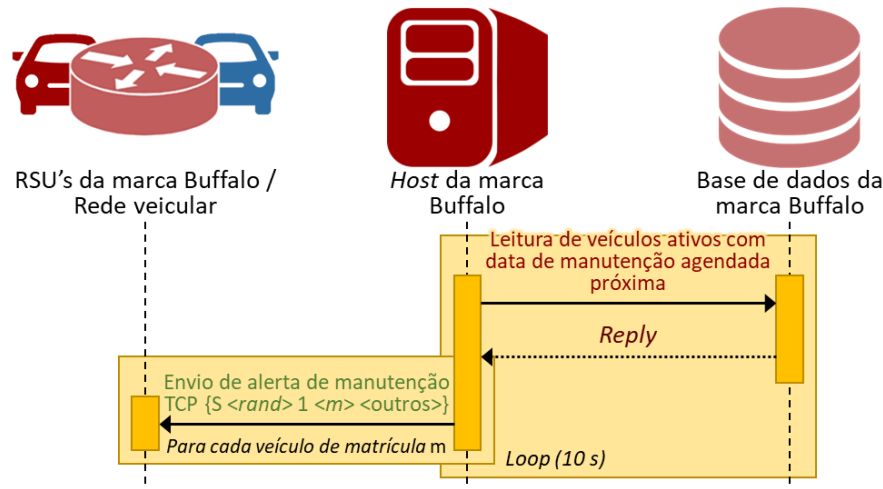


Figura 5.7 – Diagrama de sequência do envio de alertas de manutenção.

Como a marca apenas envia alertas de falta de manutenção e não do registo da mesma, para que a notificação do alerta desapareça do ecrã da aplicação veicular, o utilizador deve na mesma também fazer o registo de manutenção. Como o registo de manutenção deve ser feito com acordo mútuo entra a marca e o dono do veículo, não deve ser necessário haver uma notificação de registo enviada pela rede veicular, tendo apenas de ambas entidades registar manualmente o sucedido nas respetivas aplicações.

5.3.2 Partilha de localização

No serviço de partilha de localização, as mensagens tanto são originadas como destinadas a OBU's, portanto são encriptadas usando a chave pública do OBU de destino, o que requer a posse do certificado do destino, por ambos os participantes.

Quando o veículo A pretende partilhar a localização com B, é primeiro verificado se o certificado de B se encontra armazenado na diretoria de A, caso não esteja, é feita a troca de certificados e depois de trocados, é feito a cada 3 segundos o pedido de partilha de localização, com a seguinte estrutura de mensagem:

$S \langle rand \rangle 1 \langle destinationPlate \rangle L \langle originPlate \rangle$

- **destinationPlate** – matrícula de destino;

Encriptado:

- **originPlate** – matrícula de origem.

A espera pela resposta a este pedido encontra-se demonstrada na Figura 5.8.

```
root@n8: /home/edsantos/PTI2/network/n8

=== [ Buffalo OBU 13-37-PT ] ===

    --- [ Main menu ] ---

[1] List of active alerts
[2] <vehicle plate> Call user
[3] End location sharing
[4] Do maintenance
[5] List of stored certificates
>>>
[Location] 41.4480545828748759, -8.2964715956919899
[Scheduled maintenance] Not needed for a while
[IMTT alerts] No active alerts
[Voice call] Microphone not in use
[Voice call] Not receiving audio
[Location sharing] Waiting for 20-JU-16's response...
```

Figura 5.8 – OBU à espera da aceitação do pedido de partilha de localização.

Ao ser recebido o pedido de A, se o veículo B o aceitar, irá enviar a cada 3 segundos, as suas coordenadas geográficas, numa mensagem com a seguinte estrutura:

$S \langle rand \rangle 1 \langle destinationPlate \rangle L \langle originPlate \rangle \langle latitude \rangle \langle longitude \rangle$

Encriptado:

- **latitude** – latitude (em graus) do veículo;
- **longitude** – longitude (em graus) do veículo.

A aceitação do pedido está demonstrada na Figura 5.9.

```

root@n12: /home/edsantos/PTI2/network/n12

=== [ Buffalo OBU 20-JU-16 ] ===

--- [ Main menu ] ---

[1] List of active alerts
[2] <vehicle plate> Call user
[3] <vehicle plate> Share location with user
[4] Do maintenance
[5] List of stored certificates
>>>
[Location] 41.446903165102529, -8.2929007924376129
[Scheduled maintenance] Not needed for a while
[IMTT alerts] No active alerts
[Voice call] Microphone not in use
[Voice call] Not receiving audio
[7] Accept 13-37-PT's location sharing request

```

Figura 5.9 – Pedido de partilha de localização recebido por OBU.

Quando o veículo que fez o pedido recebe as coordenadas pedidas, as coordenadas de ambos os veículos são trocadas e são-lhes apresentadas a distância e a orientação que se encontram um do outro, como demonstrado na Figura 5.10, até alguém cancelar.

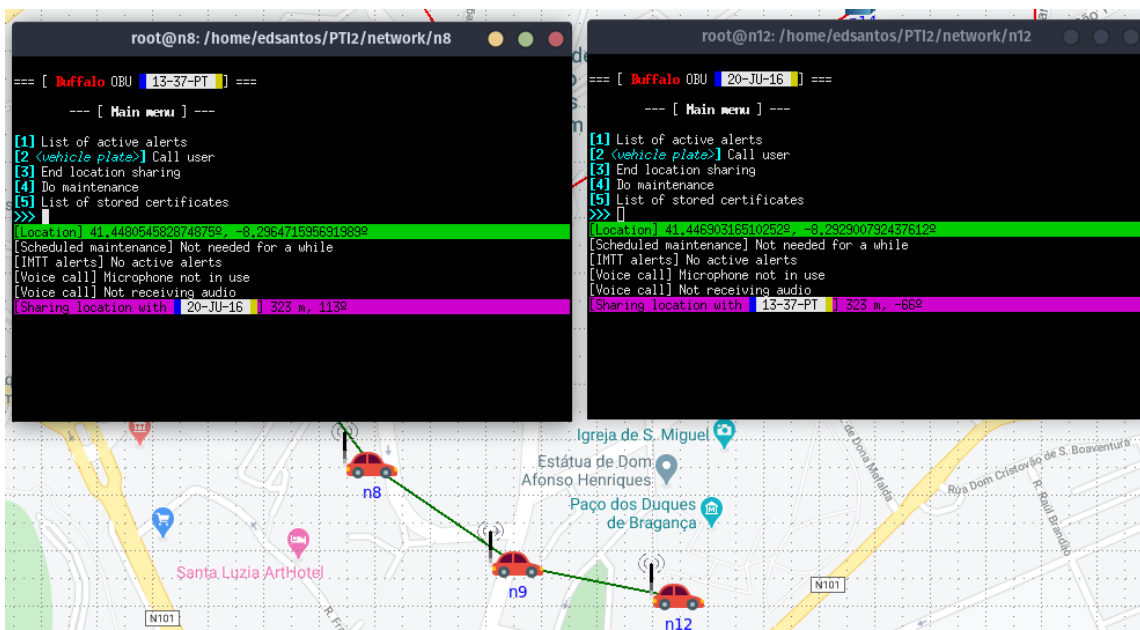


Figura 5.10 – Partilha de localização feita entre dois OBU's.

O cancelamento é feito através de uma mensagem com a seguinte estrutura:

$S \langle rand \rangle 1 \langle destinationPlate \rangle LC \langle originPlate \rangle$

Esta mensagem é enviada cinco vezes em intervalos de 1 segundo.

Na Figura 5.11 encontra-se o processo de pedido e aceitação da partilha de localização.

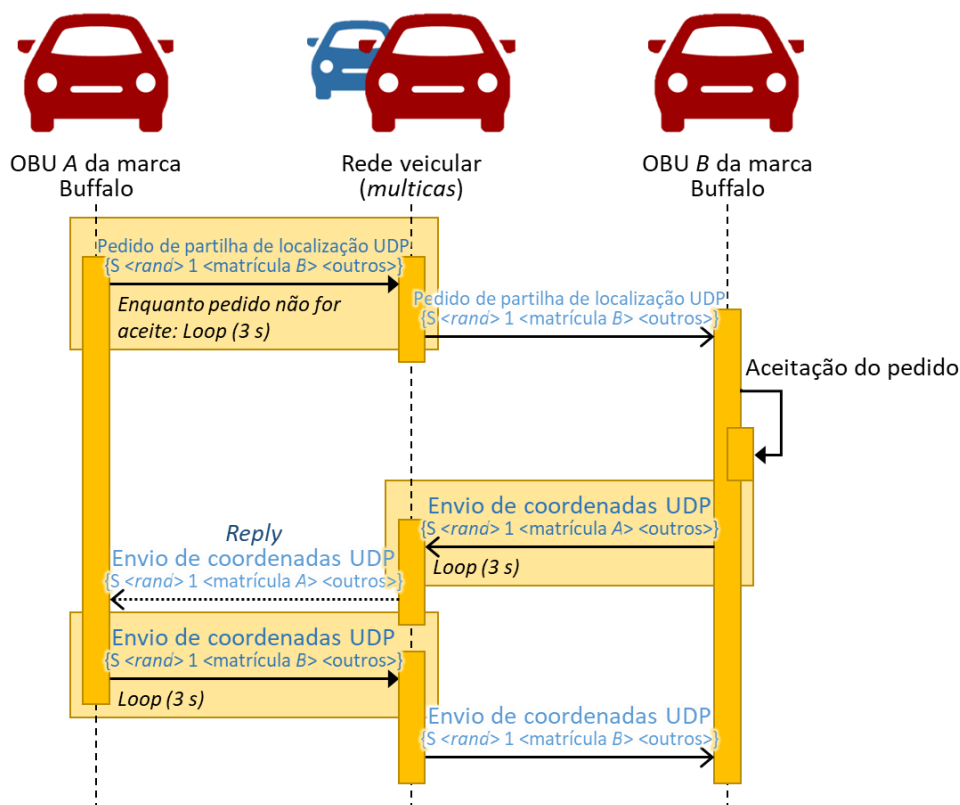


Figura 5.11 – Diagrama de sequência do pedido, aceitação e partilha de localização.

Nota: o ideal seria ter também um *timeout* para quando mensagens de partilha de localização deixam de ser recebidas, para o caso das mensagens de término da partilha serem perdidas, contudo não foi possível implementá-lo; tais razões deveram-se ao tempo e ao serviço de chamada de voz que, devido às limitações do CORE, apenas um participante envia pacotes, sendo neste detetado *timeout* por falta de receção de pacotes do segundo participante.

Nota: planeou-se usar uma interface gráfica para apresentar um mapa para os participantes com a sinalização das coordenadas dos mesmos; contudo, a visualização de mapas só é possível em *web* e não conseguimos encontrar uma forma grátis de criar uma *webview* em Java.

5.3.3 Chamada de voz

Semelhante ao serviço de partilha de localização, as chamadas de voz são à base de mensagens originadas e destinadas a OBU's, sendo também encriptadas a partir de chaves públicas. Contudo, essas não são as únicas chaves usadas para encriptar mensagens.

Devido ao extenso tamanho definido para os pacotes de áudio (20 000 bytes), não é possível usar chaves RSA para encriptar mensagens, pois tendo estas 256 bytes, não podem ultrapassar o tamanho da mensagem, e tanto a criação como a encriptação com chaves de 20 000 bytes seriam demasiado demoradas. A solução para este problema foi usar encriptação simétrica, usando chaves AES (*Advanced Encryption Standard*) de 128 bytes.

Para cada chamada, a chave de encriptação é naturalmente aleatória, tendo de ser enviada ao segundo participante da chamada no estabelecimento da mesma. Como esta chave só pode ser descoberta pelos participantes, é aqui que entra o papel da chave assimétrica dos certificados, usada para encriptar/desencriptar a chave simétrica.

Depois de ambos os participantes possuírem a chave necessária para encriptar e desencriptar os pacotes de áudio, a chamada é iniciada, sendo esses pacotes encriptados por blocos usando CBC (*Cipher Block Chaining*) com um IV (*Initialization Vector*) aleatório de 16 bits.

A estrutura da mensagem de pedido de estabelecimento de chamada é a seguinte:

$S \langle rand \rangle 1 \langle destinationPlate \rangle V \langle originPlate \rangle \langle symmetricKey \rangle$

Encriptado assimetricamente:

- **symmetricKey** – chave simétrica AES gerada para encriptar e desencriptar pacotes de áudio.

Até ser aceite, quem fez o pedido tem a mensagem de espera da Figura 5.12 e, a quem o recebe, é-lhe apresentada a mensagem da Figura 5.13.


```
root@n12: /home/edsantos/PTI2/network/n12

=== [ Buffalo OBU 20-JU-16 ] ===

    --- [ Main menu ] ---

[1] List of active alerts
[2] End call
[3] <vehicle plate> Share location with user
[4] Do maintenance
[5] List of stored certificates
>>>
[Location] 41.446903165102529, -8.2929007924376129
[Scheduled maintenance] Not needed for a while
[IMTT alerts] No active alerts
[Voice call] Waiting for 13-37-PT's response...
[Voice call] Not receiving audio
[Location sharing] Disabled
```

Figura 5.12 - OBU à espera da aceitação do pedido de chamada.

```
root@n8: /home/edsantos/PTI2/network/n8

=== [ Buffalo OBU 13-37-PT ] ===

    --- [ Main menu ] ---

[1] List of active alerts
[2] <vehicle plate> Call user
[3] <vehicle plate> Share location with user
[4] Do maintenance
[5] List of stored certificates
>>>
[Location] 41.4480545828748759, -8.2964715956919899
[Scheduled maintenance] Not needed for a while
[IMTT alerts] No active alerts
[6] Accept 20-JU-16's voice call
[Voice call] Not receiving audio
[Location sharing] Disabled
```

Figura 5.13 – Pedido de chamada recebido por OBU.

Depois da chamada ser aceite, a estrutura dos pacotes de áudio é a seguinte:

$S \langle rand \rangle 1 \langle destinationPlate \rangle V \langle originPlate \rangle \langle audio \rangle$

Encriptado simetricamente:

- **audio** – pacote de áudio encriptado com a chave simétrica.

O processo de término da chamada é semelhante ao término da partilha de localização, sendo a estrutura da mensagem a seguinte:

$S \langle rand \rangle 1 \langle destinationPlate \rangle VC \langle originPlate \rangle$

Na Figura 5.14 encontra-se o processo de pedido e aceitação da chamada de voz.

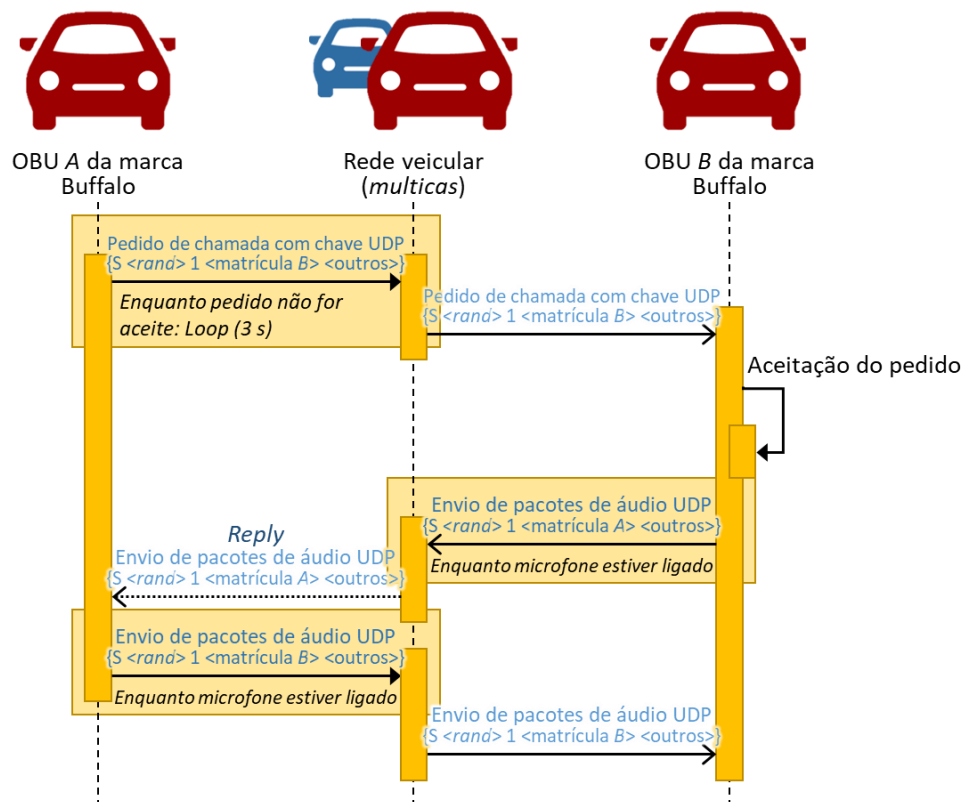


Figura 5.14 – Diagrama de sequência do pedido, aceitação e funcionamento da chamada.

Na Figura 5.15 e Figura 5.16 encontram-se os indicadores da intensidade do áudio de cada pacote enviado e recebido (calculada a partir da média dos bytes do pacote), atualizado constantemente.

```
root@n8: /home/edsantos/PTI2/network/n8

=== [ Buffalo OBU 13-37-PT ] ===

    --- [ Main menu ] ---

[1] List of active alerts
[2] End call
[3] <vehicle plate> Share location with user
[4] Do maintenance
[5] List of stored certificates
>>>
[Location] 41.4480545828748759, -8.2964715956919899
[Scheduled maintenance] Not needed for a while
[IMTT alerts] No active alerts
[Voice call] #####
[ 20-JU-16 ] #####
[Location sharing] Disabled
```

Figura 5.15 – Intensidade de um pacote de áudio enviado por um OBU.

```
root@n12: /home/edsantos/PTI2/network/n12

=== [ Buffalo OBU 20-JU-16 ] ===

    --- [ Main menu ] ---

[1] List of active alerts
[2] End call
[3] <vehicle plate> Share location with user
[4] Do maintenance
[5] List of stored certificates
>>>
[Location] 41.446903165102529, -8.2929007924376129
[Scheduled maintenance] Not needed for a while
[IMTT alerts] No active alerts
[Voice call] #####
[ 13-37-PT ] #####
[Location sharing] Disabled
```

Figura 5.16 – Intensidade de um pacote de áudio recebido por um OBU.

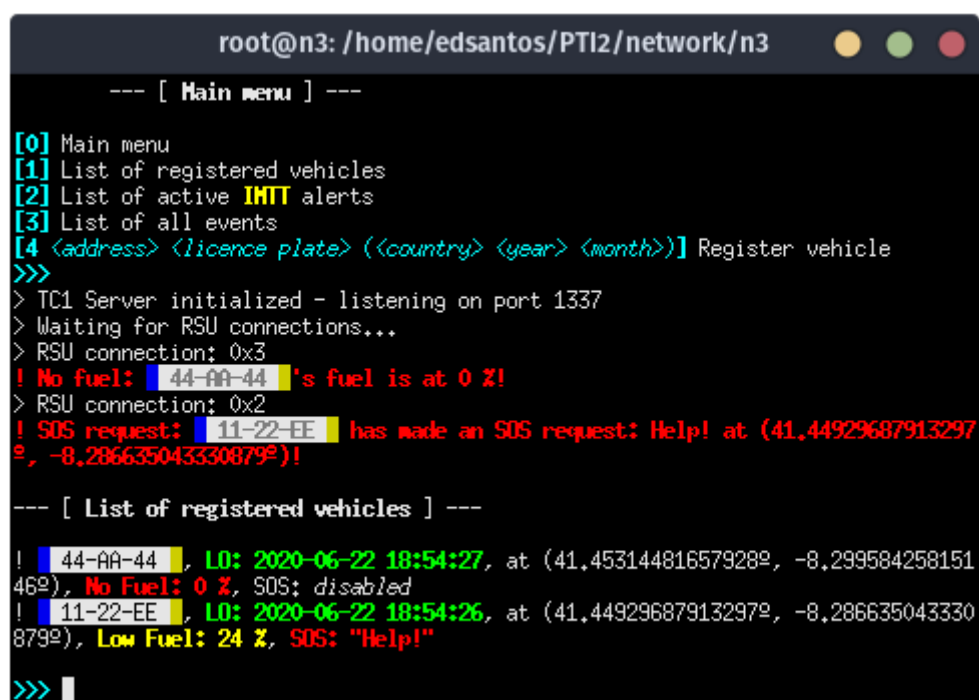
Nota: como referido anteriormente e visível na Figura 5.15 e Figura 5.16, no CORE, apenas um participante pode enviar áudio e o outro recebê-lo; isto deve-se a ambos os participantes acederem ao mesmo *hardware* que é o microfone e as colunas do computador *host*, e não podem ser usados por várias instâncias ao mesmo tempo. Contudo, este sistema foi testado fora da topologia em dois computadores diferentes (sem *multi-cast*) e confirmou-se a possibilidade de ambos enviarem e receberem áudio ao mesmo tempo.

6 Marca Cheetah #2

A marca Cheetah é uma marca notável pelos seus serviços baseados em localização geográfica e acesso à Internet. Os serviços desta marca são: *geocoding*, alertas de baixo combustível com posto mais próximo e pedidos de SOS.

6.1 Monitorização de veículos

A monitorização de veículos é feita pelo *host* da marca, em que a listagem dos mesmos, como se pode ver na Figura 6.1, contém a respetiva matrícula, *timestamp* da última mensagem de *keep alive* recebida (**LO** – *Last time Online*), latitude e longitude da localização do veículo, percentagem de combustível e estado e mensagem de SOS.



```
root@n3: /home/edsantos/PTI2/network/n3
--- [ Main menu ] ---
[0] Main menu
[1] List of registered vehicles
[2] List of active IHIT alerts
[3] List of all events
[4] <address> <licence plate> (<country> <year> <month>)] Register vehicle
>>>
> TC1 Server initialized - listening on port 1337
> Waiting for RSU connections...
> RSU connection: 0x3
! No fuel: 44-AA-44's fuel is at 0 %!
> RSU connection: 0x2
! SOS request: 11-22-EE has made an SOS request: Help! at (41.44929687913297
e, -8.286635043330879e)!

--- [ List of registered vehicles ] ---
! 44-AA-44, LO: 2020-06-22 18:54:27, at (41.45314481657928e, -8.299584258151
46e), No Fuel: 0 %, SOS: disabled
! 11-22-EE, LO: 2020-06-22 18:54:26, at (41.44929687913297e, -8.286635043330
879e), Low Fuel: 24 %, SOS: "Help!"
>>>
```

Figura 6.1 – Monitorização de veículos da marca Cheetah.

Nesta lista são realçados diversos estados:

- **Veículo online:** LO de há menos de 60 segundos;
- **Baixo combustível:** percentagem de combustível inferior a 30 %;
- **Sem combustível:** depósito de combustível a 0 %.
- **SOS ativo:** mensagem de SOS definida pelo veículo.

Nesta marca, a mensagem de *keep alive* tem a seguinte estrutura:

S <rand> 2 brand 0 <licencePlate> <latitude> <longitude> <fuel> (<sos>)

Encriptado:

- **fuel** – percentagem de combustível;
- **sos** – mensagem de SOS.

6.2 Serviços

6.2.1 Geocoding – informação da localização

O serviço de *geocoding* consiste em obter o nome da rua e a localidade onde se encontra um veículo sendo para tal usada a Geocoding API [10] da Google Maps [11], que requer o acesso à Internet por parte do *host* da marca.

Na topologia do CORE, é geralmente usada a ferramenta RJ45 para aceder à Internet através da interface de rede usada pelo computador *host*; contudo, este método impede o acesso à Internet pelo computador, impossibilitando a apresentação *online* deste sistema. A solução usada foi semelhante ao serviço de localização: usar um *script* em Python a correr no computador *host* à espera de mensagens UDP. Neste caso, a mensagem do pedido contém a latitude e longitude do veículo e o *script* usa essas coordenadas para fazer o pedido HTTP (*Hypertext Transfer Protocol*) GET à Geocoding API, de modo a obter a informação pretendida, sendo reencaminhada para o *host* da marca que fez o pedido.

Os pedidos HTTP são feitos ao seguinte URL (*Uniform Resource Locator*):

<https://maps.googleapis.com/maps/api/geocode/json?latlng=<latitude>,<longitude>&key=<apiKey>>

- **latitude** – latitude (em graus) do veículo;
- **longitude** – longitude (em graus) do veículo;
- **apiKey** – chave providenciada pela Google para fazer pedidos às suas API's.

A cada 10 segundos, é determinada a partir da base de dados, a lista de veículos ativos nos últimos 60 segundos, sendo para os selecionados determinado o seu *geocode* que é enviado numa mensagem para os respetivos veículos, seguindo a seguinte estrutura:

$S \langle rand \rangle 2 \langle licencePlate \rangle G C \langle geocode \rangle$

Encriptado:

- **geocode** – nome da localidade e rua onde o veículo se encontra.

Na Figura 6.2 encontra-se o processo de determinação e envio do *geocode*.

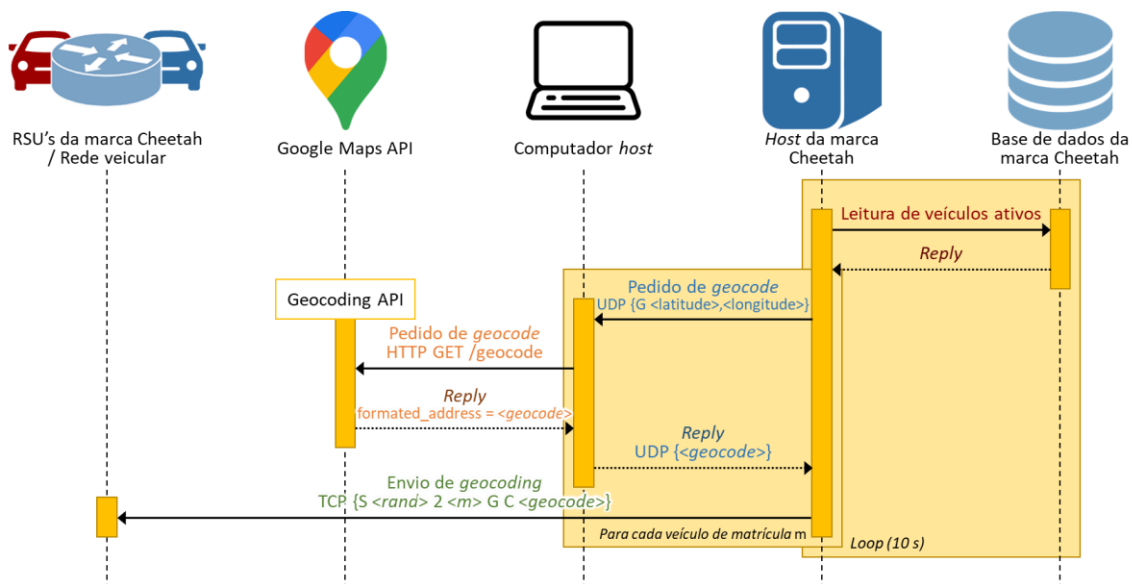


Figura 6.2 – Diagrama de sequência da determinação e envio do *geocode*.

Na Figura 6.3 encontra-se a visualização do *geocode* na aplicação veicular.

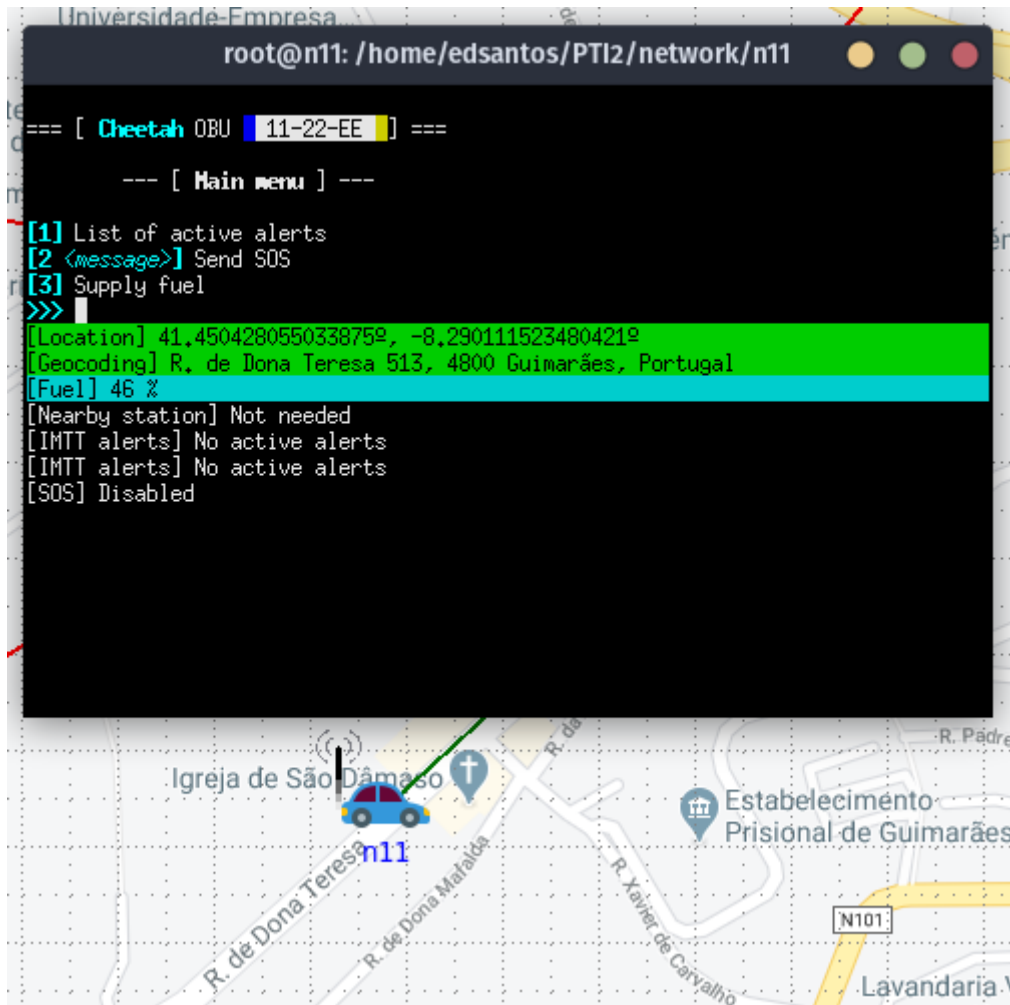


Figura 6.3 – Geocode visualizado na aplicação veicular.

Como se pode ver na figura, o nome da rua apresentado corresponde à rua indicada no mapa – “R. de Dona Teresa”.

6.2.2 Alertas de baixo combustível e posto mais próximo

Na monitorização a cada 10 segundos dos veículos, os que se encontram ativos e com depósito de combustível inferior a 30 % são alertados, sendo-lhes dada informação acerca do posto de combustível mais próximo – o nome do posto e as coordenadas geográficas.

À semelhança do serviço de *geocoding*, este serviço usa também a Google Maps API para obter a informação pretendida; desta vez, é usada a Places API [12] para obter a lista de postos de combustível nas proximidades das coordenadas do veículo, ordenada crescentemente por distância.

O URL usado é o seguinte:

https://maps.googleapis.com/maps/api/place/nearbysearch/json?location=<latitude>,<longitude>&rankby=distance&type=gas_station&key=<apiKey>

A informação obtida é encaminhada para os respetivos veículos na mesma mensagem que contém o *geocode*, e em vez de ter a estrutura referida anteriormente, tem uma semelhante:

$S \langle rand \rangle 2 \langle licencePlate \rangle G S \langle geocode \rangle || \langle latitude \rangle \langle longitude \rangle \langle gasStation \rangle$

Encriptado:

- **latitude** – latitude (em graus) do posto de combustível mais próximo;
- **longitude** – longitude (em graus) do posto de combustível mais próximo;
- **gasStation** – nome do posto do combustível mais próximo.

Como referido, este serviço complementa o serviço de *geocoding* no envio de mensagens, sendo enviadas segundo a Figura 6.4.

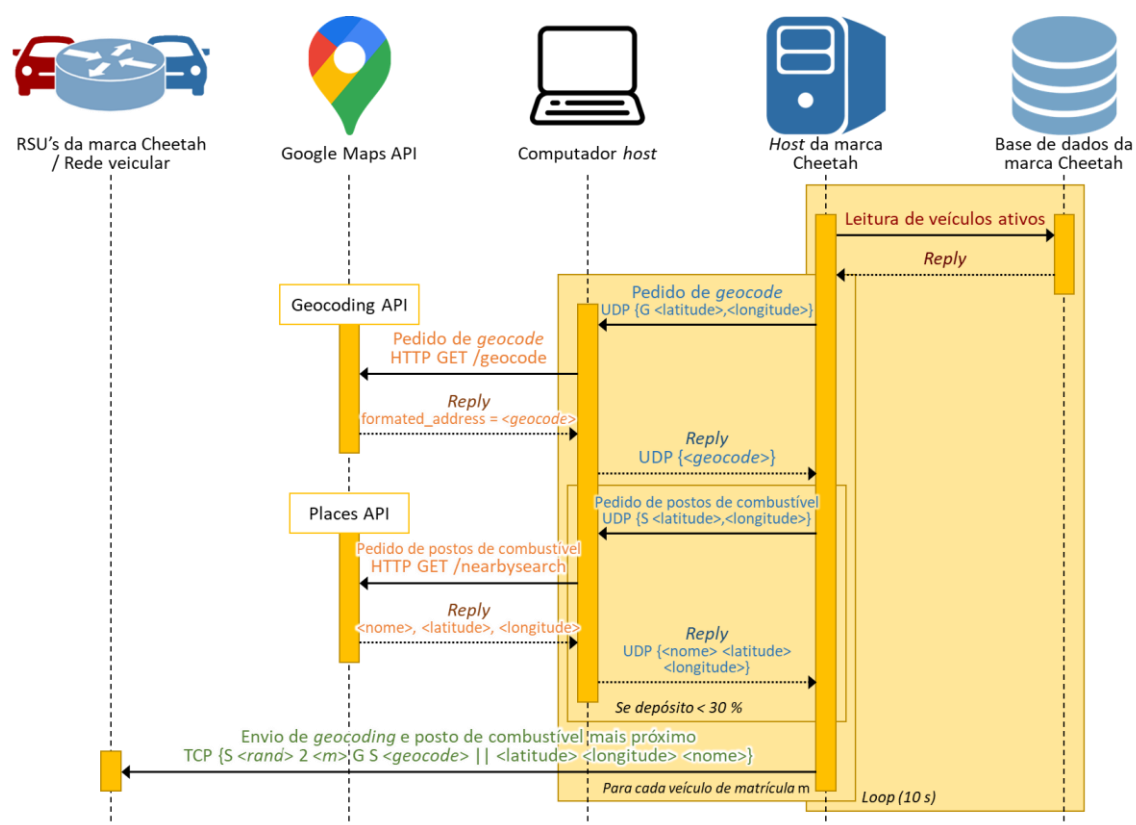


Figura 6.4 – Diagrama de sequência da obtenção do posto de combustível mais próximo para concatenar ao *geocode*.

Nos *keep alive*s recebidos pela marca, ao ser armazenada a percentagem de combustível e verificado que ao contrário do valor previamente armazenado, se encontra com valor = 0, o *host* é notificado, sendo gerado um evento na base de dados com os parâmetros da Tabela 6.1.

Tabela 6.1 – Estrutura do alerta de combustível vazio armazenado na tabela de eventos.

Evento	type	source	destination	ttr	latitude, longitude	radius	message
Registo de manutenção	"F"	matrícula do veículo	"brand"			-	

Na Figura 6.5 encontra-se o aviso do posto de combustível mais próximo, assim como a distância e orientação ao mesmo.

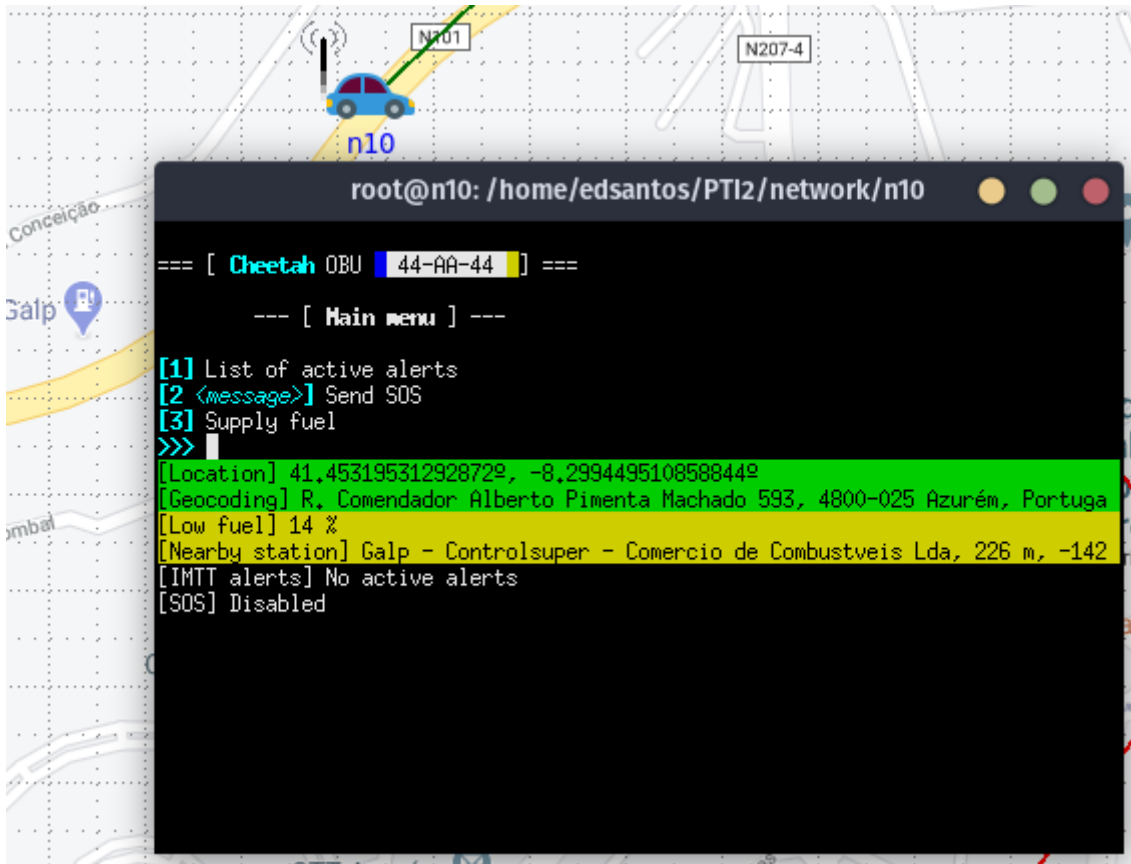


Figura 6.5 – Posto de combustível mais próximo apresentado na aplicação veicular.

Como se pode confirmar no mapa, o posto mais próximo é a Galp [13].

No caso de combustível vazio, o mesmo facto é realçado, como se pode ver na Figura 6.6.

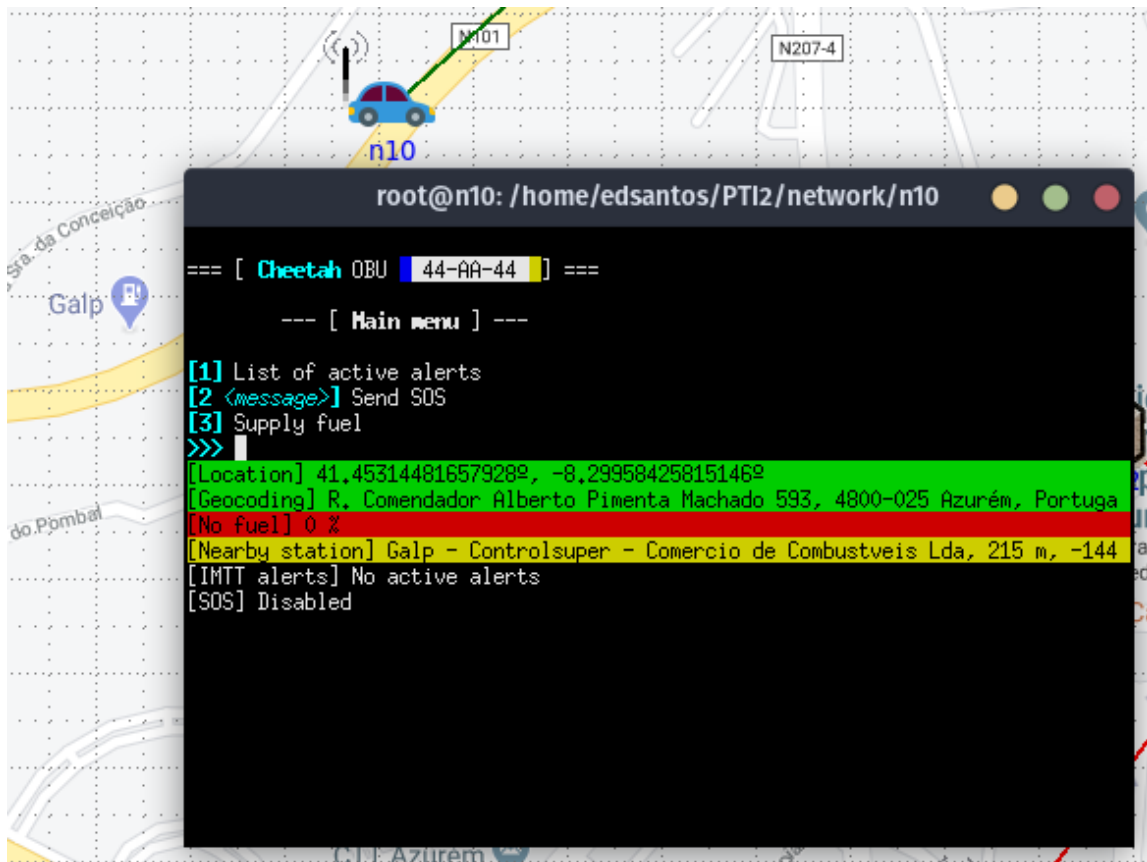


Figura 6.6 – Combustível vazio indicado na aplicação veicular.

Como se pôde ver anteriormente na Figura 6.1, a marca é notificada em tal situação.

A percentagem de combustível num veículo é reduzida de acordo com o seu movimento e para demonstrar os alertas para diversas percentagens, o decréscimo é propositadamente enorme, tendo um consumo de 1 %/10 metros.

Para abastecer, a aplicação veicular dispõe no menu principal duma funcionalidade para tal que ao ser seleccionada, a percentagem de combustível passa a ser 100 % (sem qualquer custo!).

6.2.3 Pedidos de SOS

As aplicações veiculares têm também a função de fazer pedidos SOS com uma mensagem personalizada.

Quando é definida uma mensagem de SOS, a mesma é concatenada à mensagem de *keep alive*.

Quando o *host* da marca recebe um *keep alive*, é verificado se a mensagem de alerta é diferente da armazenada (ativação, alteração e desativação) e neste caso, são invocados uma notificação e um novo evento com os parâmetros da Tabela 6.2.

Tabela 6.2 – Estrutura do pedido de SOS armazenado na tabela de eventos.

Evento	type	source	destination	tth	latitude, longitude	radius	message
Registo de manutenção	"S"	matrícula do veículo	"brand"	-	-		mensagem de SOS

Ainda na mesma condição, se adicionalmente a mensagem conter a palavra “accident”, o *host* da marca envia para o do IMTT um pedido de alerta com as coordenadas do veículo, alcance de 100 metros e mensagem “Accident”, para que o IMTT lance um alerta geral com os parâmetros definidos e um TTL de 5 horas.

A mensagem recebida pelo alerta tem a seguinte estrutura:

A <latitude> <longitude> 100 Accident

O processo deste alerta dinâmico encontra-se esquematizado na Figura 6.7.

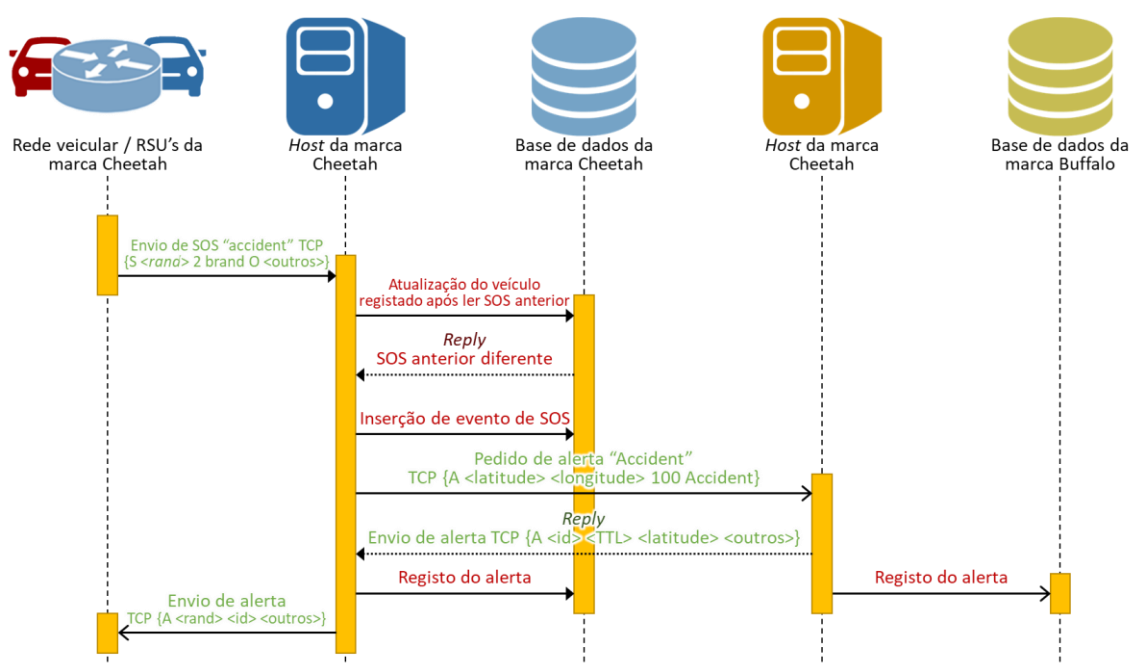


Figura 6.7 – Diagrama de sequência do alerta de acidente lançado pelo IMTT após pedido da marca.

Na Figura 6.8 encontra-se o lançamento do pedido de SOS com a mensagem “I got an accident!” que provocou o alerta “Accident”.

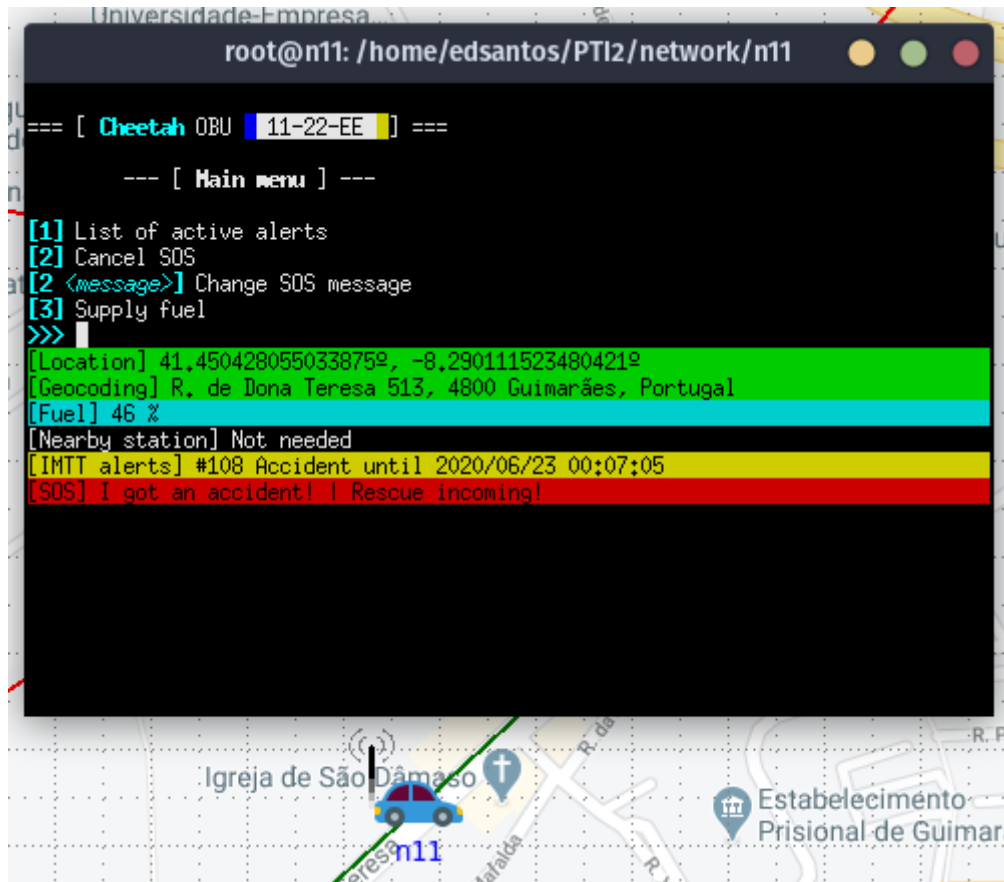


Figura 6.8 – Pedido de SOS feito pelo OBU com alerta recebido pelo IMTT.

Ao colocar outro veículo (independentemente da marca) a 186 metros do epicentro do acidente (coordenadas do veículo que fez o pedido de SOS), como o alcance do acidente é de 100 metros, a distância do acidente é 86 metros, como se pode ver na Figura 6.9.

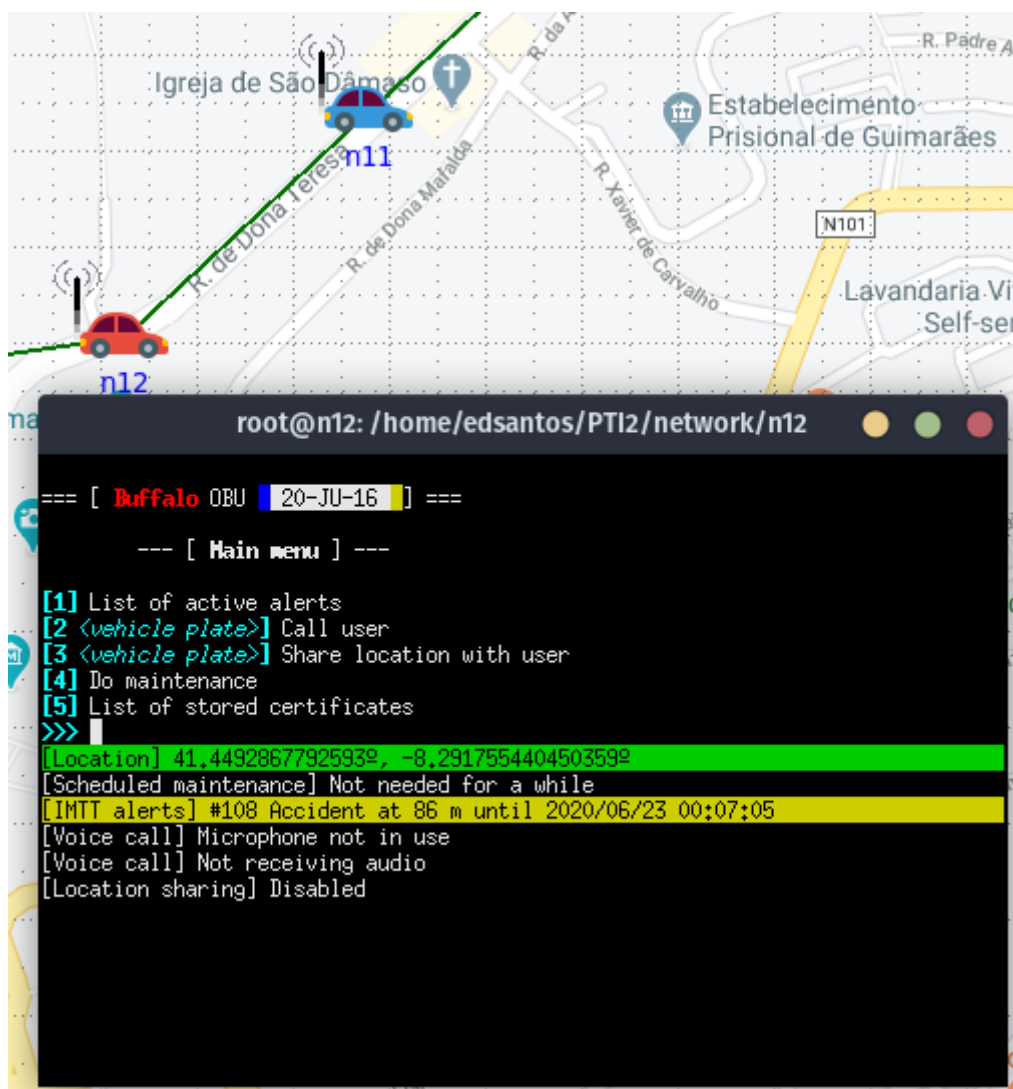


Figura 6.9 – Notificação de alerta IMTT num veículo nas proximidades do acidente.

Na Figura 6.10 encontram-se as notificações da marca ao receber o pedido de SOS, o alerta do IMTT provocado pelo mesmo, e o cancelamento do pedido.

```
root@n3: /home/edsantos/PTI2/network/n3

=== [ Cheetah Host ] ===

> Connecting to IMTT's TCP Server @2001:690:2280:820::10:1336...
> Successfully connected to IMTT's Server @2001:690:2280:820::10:1336

--- [ Main menu ] ---

[0] Main menu
[1] List of registered vehicles
[2] List of active IMTT alerts
[3] List of all events
[4] <address> <licence plate> (<country> <year> <month>)] Register vehicle
>>>
> TCP Server initialized - listening on port 1337
> Waiting for RSU connections...
> RSU connection: 0x3
> RSU connection: 0x2
! SOS request: 11-22-EE has made an SOS request: I got an accident! at (41,4
50428055033875º, -8,290111523480421º)!
> IMTT alert: #108 Accident at (41,450428055033875º, -8,290111523480421º) ±100 m
until 2020/06/23 00:07:05
> SOS cancelation: 11-22-EE has canceled its SOS request
```

Figura 6.10 – Pedido de SOS (e cancelamento) e alerta IMTT recebidos pela marca.

Nota: durante a apresentação final, esta funcionalidade não pôde ser demonstrada devido a uma simples linha de código em falta. A tabela de eventos foi concluída no fim de semana antes da apresentação, assim como foi programada a função de armazenar pedidos de SOS nessa tabela. Infelizmente não foi testado o comportamento do registo de pedidos de SOS o que fez com que o problema não fosse descoberto. A linha de código referida trata-se da definição da *query* MySQL para inserção na tabela de eventos que estava vazia, mas os métodos do Java estavam a tentar preencher valores nessa *query* vazia, provocando o *crash* do programa, como foi visto na apresentação. Como este serviço é um dos mais importantes na nossa opinião, pois não só inclui pedidos de SOS como também é um exemplo de lançamento de alertas dinâmico e automático, achamos que é necessário que os professores testemunhem o seu funcionamento, e devido a um simples erro, não foi possível o fazerem em direto. Para compensar, foi gravado um vídeo com o funcionamento do serviço, anexado no ZIP submetido. **Reparamos que o vídeo não reproduz áudio no Windows, apenas em Linux e Android.**

7 Funcionalidades planeadas não implementadas

Houve várias funcionalidades planeadas que não foram implementadas devido a falta de tempo, limitações do CORE e algumas complexidades. Tais funcionalidades incluem:

- **Rádio** – este serviço foi descartado poucos dias antes da apresentação final devido a termos subestimado a complexidade necessária que requeria um tempo extra para o concluirmos. Os problemas que surgiram foram no tamanho dos pacotes necessários para uma transmissão com qualidade perceptível que requeriam centenas de KBytes, o que não é possível em pacotes UDP, limitados a 60 KBytes. Uma solução seria a divisão desses pacotes, mas isso aumentaria imenso a perda dos mesmos e o áudio recebido seria impercetível. Outra solução complementar requeria o uso de *buffers* mas isso poderia atrasar e até mesmo abrandar a reprodução e eventualmente esse *buffer* ficaria cheio.
- **Alertas meteorológicos** – este serviço foi descartado poucas semanas antes da apresentação final devido a falta de tempo. Este tipo de alerta seria um dos tipos dinâmicos e automáticos, em que seriam obtidos os dados meteorológicos de todos os distritos do país através da OpenWeatherMap API [14] a cada 6 horas, sendo enviado um alerta para cada distrito contendo um TTL de 6 horas, latitude e longitude do centro do distrito, raio do mesmo e mensagem contendo o estado meteorológico e a temperatura atuais.
- **Anúncios** – este seria um serviço simples de implementar, mas foi descartado pois outros grupos reservaram-no. Contudo, pode-se dizer que está implementado ao se fazer um anúncio manualmente nos alertas pelo IMTT.
- **Limites de velocidade** – outro serviço descartado devido a estar reservado por outro grupo. Este serviço seria simples de implementar juntamente com o de *geocoding* a partir de outra API da Google Maps – Roads API [15], que dadas umas coordenadas, é devolvido o limite de velocidade da rua onde se encontra. Este serviço também seria complementado com a velocidade instantânea de cada veículo, calculada a partir das suas coordenadas e distância percorrida em cada instante. A velocidade do veículo seria contida no *keep alive* e notificaria a marca caso os limites fossem ultrapassados.
- **Localização a partir de triangulação com os nós da rede** – este serviço seria uma alternativa ao sistema implementado e num sistema real, seria usado em zonas com falta de captação de satélite, pois usaria a localização dos veículos no alcance e a distância dos mesmos para determinar a localização pretendida. Como os RSU's são fixos, estes teriam as suas coordenadas *hardcoded* e iriam servir de referência para os veículos da rede. Esta funcionalidade não foi implementada pois não conseguimos calcular a distância entre dois nós; possivelmente devido à falta de um RTT apropriado para calcular a distância (o comando *ping* devolve sempre valores na ordem dos 40 ms), pensamos que a solução para este problema seria o uso de Wi-Fi RTT (IEEE 802.11mc) [16] num sistema real.

- **Comunicação por DNS** – a ideia deste serviço surgiu na semana anterior à apresentação final, o que não deu tempo para implementá-la. Este serviço consiste em dar alguma utilidade ao endereço DNS atribuído ao veículos em que quando um veículo *A* pretende comunicar com *B*, poderia ser usado o endereço DNS de *B* como endereço de destino em vez de *multicast*; isto requeria que os veículos estivessem sempre no alcance um do outro mas seria uma forma de evitar tantos parâmetros no cabeçalho e encriptação, pois a mensagem seria apenas direcionada para o veículo correspondido. Seria apenas uma alternativa opcional, à escolha do utilizador, para trocar alcance por desempenho.
- **Alertas e cálculo do nível de trânsito** – este serviço teria por base uma funcionalidade do projeto de PTI 1 do semestre passado, em que as coordenadas de todos os veículos seriam enviadas para o IMTT, de modo a determinar as zonas com trânsito congestionado, para automaticamente alertas os veículos nas proximidades dessas zonas.
- **“Buzina”** – este serviço seria comum a todas as marcas e consiste nos OBU’s poderem lançar um alerta semelhante ao propósito das buzinas para notificar todos os veículos no seu alcance da sua proximidade.

7.1 *Mobility script*

Planeou-se desde o início usar *mobility script* para demonstração do funcionamento automático da rede, e até mesmo testou-se um pequeno protótipo. Contudo, não houve tempo suficiente para o implementarmos; além disso seria necessário um mapa com poucas curvas e o de Guimarães não seria o indicado para tal.

8 *Software utilizado*

Para o desenvolvimento deste projeto, foi usado o seguinte *software*, bibliotecas externas e API’s:

- **CORE 4.6** [1] **em VirtualBox** [17] – criação da topologia final e seus endereçamentos, configuração dos nós (incluindo BGP e OSPF) e serviços de DNS e MySQL a correr na topologia;
- **CORE 6.3** [1] – serviços das marcas e de localização;
- **BIND 9** [7] – serviço de DNS
- **Visual Studio Code** [18] – programação Java e Python;
- **MySQL Connector/J 8.0.19** [19] – biblioteca Java para MySQL.
- **Google Maps** [11] – API’s e mapa de Guimarães (Figura 8.1) para a topologia secundária, usada para desenvolvimento dos serviços.

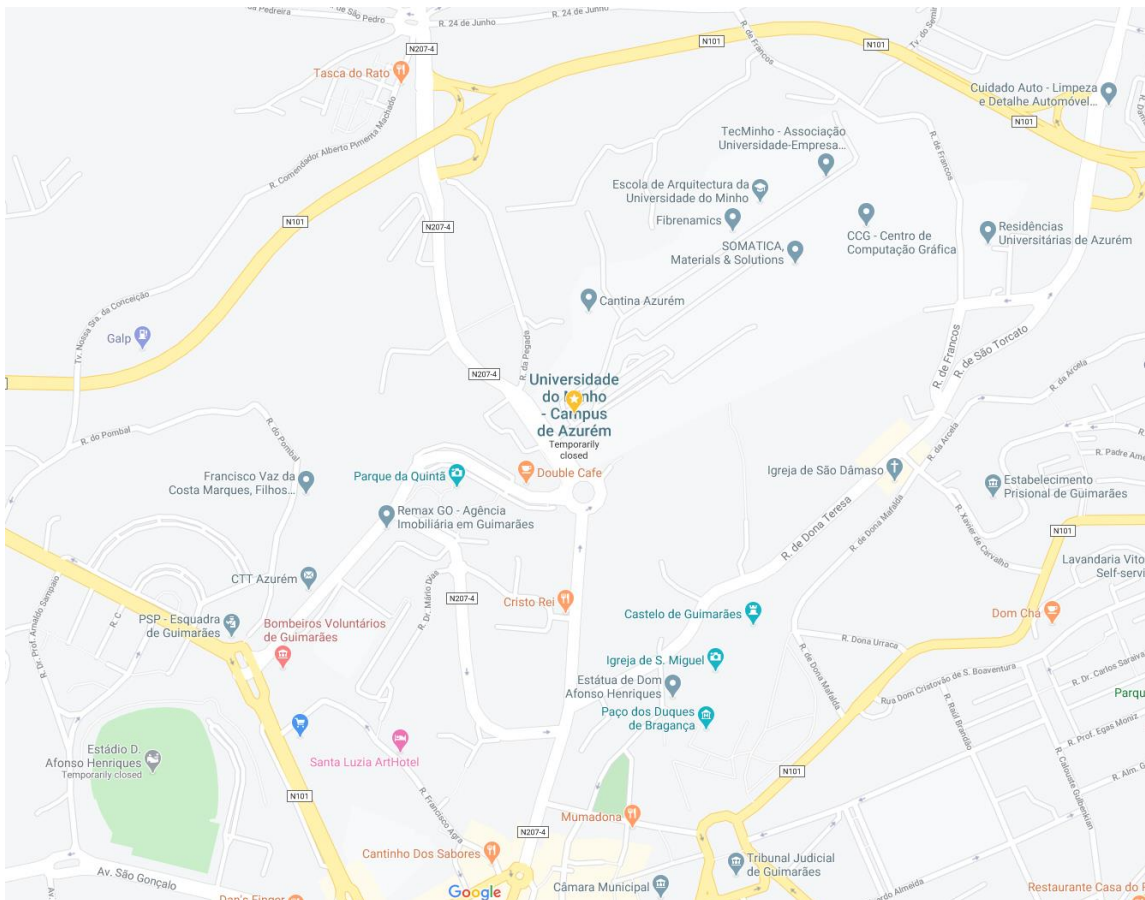


Figura 8.1 – Mapa da topologia secundária.

9 Conclusão

Neste projeto foram aplicados e adquiridos novos conhecimentos acerca das funcionalidades do CORE e foram postos à prova conhecimentos de UC's de semestres anteriores, como emulação de redes no CORE, encaminhamento OSPF, BGP e endereçamento IPv6 (Redes de Computadores II), desenvolvimento de bases de dados (Tecnologias de Bases de Dados), desenvolvimento de arquiteturas Cliente/Servidor TCP *multi-thread* em Java (Sistemas Distribuídos) e encriptação e certificados (Criptografia).

Em relação às fases anteriores, foi melhorado o endereçamento da topologia e o sistema de bases de dados SQLite [6] foi substituído pelo MySQL.

Pensamos que este projeto teve um tema muito interessante pois é algo que apesar de não ser muito comum atualmente, muito provavelmente será uma norma na área veicular num futuro não muito distante. O meio de comunicação desenvolvido entre os veículos é compatível não só com os serviços que implementamos e planeamos, mas também muitos mais, principalmente na condução autónoma pois a informação recebida e enviada para veículos nas proximidades é útil para que os veículos tomem as ações necessárias de acordo com essa informação (abrandamento de velocidade quando há veículos na proximidade, aceleração em caso contrário, etc.).

Surgiram algumas dificuldades, principalmente relacionadas com o CORE, como a aquisição de coordenadas GPS cuja solução foi obtida graças à comunidade de utilizadores e *developers* do CORE através do servidor de Discord oficial [20]. As restantes dificuldades foram previamente mencionadas, sendo as mesmas resolvidas ou pelo menos remediadas.

Achamos que estas dificuldades se deveram a limitações do próprio CORE ou até mesmo por emular uma rede num só computador (chamadas de voz, instâncias de MySQL, Internet, GPS, interfaces gráficas, etc.); contudo, achamos que os serviços desenvolvidos funcionariam devidamente em redes reais onde todos os OBU's e *hosts* são máquinas independentes umas das outras.

Finalmente, achamos que concluímos este projeto com sucesso. Mas como referido, um projeto neste tema nunca está concluído, sendo sempre compatível com inúmeros serviços e funcionalidades diferentes.

“A arte nunca está terminada, é apenas abandonada.” – Leonardo Da Vinci

10 Referências

- [1] Nrl.navy.mil. n.d. *Common Open Research Emulator (CORE) | Networks and Communication Systems Branch*. [online] Disponível em: <https://www.nrl.navy.mil/itd/ncs/products/core> [Acedido a 11/02/2020].
- [2] Grandtheftwiki.com. 2006. *Buffalo - Grand Theft Wiki, The GTA Wiki*. [online] Disponível em: <https://www.grandtheftwiki.com/Buffalo> [Acedido a 18/02/2020].
- [3] Grandtheftwiki.com. 2008. *Cheetah - Grand Theft Wiki, The GTA Wiki*. [online] Disponível em: <https://www.grandtheftwiki.com/Cheetah> [Acedido a 18/02/2020].
- [4] Khanna, S., 2012. *Understanding Ipv6 EUI-64 Bit Address*. [online] Community.cisco.com. Disponível em: <https://community.cisco.com/t5/networking-documents/understanding-ipv6-eui-64-bit-address/ta-p/3116953> [Acedido a 11/03/2020].
- [5] Nongnu.org. n.d. *Quagga*. [online] Disponível em: <https://www.nongnu.org/quagga/docs/quagga.html> [Acedido a 18/02/2020].
- [6] Sqlite.org. n.d. *Sqlite Home Page*. [online] Disponível em: <https://www.sqlite.org/index.html> [Acedido a 05/03/2020].
- [7] Isc.org. n.d. *Internet Systems Consortium*. [online] Disponível em: <https://www.isc.org/bind/> [Acedido a 18/02/2020].
- [8] Ellingwood, J., 2018. *How To Configure BIND As A Private Network DNS Server On Debian 9 | Digitalocean*. [online] Digitalocean.com. Disponível em: <https://www.digitalocean.com/community/tutorials/how-to-configure-bind-as-a-private-network-dns-server-on-debian-9> [Acedido a 07/03/2020].

- [9] Guoan, X., 2019. *Set Up BIND Authoritative DNS Server On Debian 10 Buster*. [online] LinuxBabe. Disponível em: <https://www.linuxbabe.com/debian/authoritative-dns-server-debian-10-buster-bind9> [Acedido a 07/03/2020].
- [10] Google Developers. (n.d.). *Get Started | Geocoding API | Google Developers*. [online] Disponível em: <https://developers.google.com/maps/documentation/geocoding/start> [Acedido a 21/02/2020].
- [11] Google Developers. n.d. *Google Maps Platform | Google Developers*. [online] Disponível em: <https://developers.google.com/maps/documentation> [Acedido a 21/02/2020].
- [12] Google Developers. n.d. *Place Search | Places API | Google Developers*. [online] Disponível em: <https://developers.google.com/places/web-service/search> [Acedido a 13/05/2020].
- [13] Galp.com. n.d. *Energia Cria Energia | Galp*. [online] Disponível em: <https://www.galp.com/corp/pt/> [Acedido a 23/06/2020].
- [14] Openweathermap.org. n.d. *Weather API - Openweathermap*. [online] Disponível em: <https://openweathermap.org/api> [Acedido a 23/03/2020].
- [15] Google Developers. n.d. *Speed Limits | Roads API | Google Developers*. [online] Disponível em: <https://developers.google.com/maps/documentation/roads/speed-limits> [Acedido a 03/04/2020].
- [16] En.wikipedia.org. n.d. *IEEE 802.11Mc*. [online] Disponível em: https://en.wikipedia.org/wiki/IEEE_802.11mc [Acedido a 26/02/2020].
- [17] Virtualbox.org. n.d. *Oracle VM Virtualbox*. [online] Disponível em: <https://www.virtualbox.org/> [Acedido a 20/03/2020].
- [18] Code.visualstudio.com. (n.d.). *Visual Studio Code - Code Editing. Redefined*. [online] Disponível em: <https://code.visualstudio.com/> [Acedido a 11/02/2020].
- [19] Dev.mysql.com. 2020. *Mysql :: Mysql Connector/J 8.0 Release Notes :: Changes In Mysql Connector/J 8.0.19 (2020-01-13, General Availability)*. [online] Disponível em: <https://dev.mysql.com/doc/relnotes/connector-j/8.0/en/news-8-0-19.html#:~:text=19%20is%20the%20latest%20General,and%20implementations%20the%20X%20DevAPI>. [Acedido a 28/03/2020].
- [20] Discord. n.d. *Join The CORE Development Discord Server!*. [online] Disponível em: <https://discord.com/invite/AKd7kmP> [Acedido a 10/02/2020].
- [21] core. n.d. *Using The Grpc API*. [online] Disponível em: <http://core-reemu.github.io/core/grpc.html> [Acedido a 03/03/2020].